

基于两阶段随机规划与情景模型预测控制的微网电力调控策略

摘 要

非孤岛式微网的购电决策受到供需波动、预报更新与电价变化的共同影响。本文在统一的能量平衡和储能状态约束下，依次建立确定性调度、随机日前购电与日内滚动调整模型，以保障供电并降低购电费用。数据分析显示，供需存在日内错位、周周期与月度差异，为储能调度和情景构造提供依据。

针对问题一，考虑典型日供需与电价的时序错位，以全天购电费用最小为目标，建立**购电与储能联合线性规划模型**，协调储能的跨时段能量分配。求得最小日购电费用 35118.60 元，较无储能基准降低 **26.92%**，并实现光伏全额消纳；进一步得到等价最优日初储电量区间 [8550, 8700] kWh，据此选取代表方案。

针对问题二，考虑负载与光伏的不确定性，建立**两阶段随机线性规划模型**，联合优化日前购电及事后储能调度与紧急购电。负载七日同刻相关系数为 0.9722，且周五、六日总负载偏低，据此采用周同期负载与净负荷日总量、前三日光伏日总量均值等特征进行**高斯核加权**。该特征组合采用平均能量分数进行效果验证，发现较等权基线降低 **58.706%**。2025 年 2 至 12 月结算费用为 **1699.85** 万元，紧急购电量占计划购电量的 **0.256%**。为检验跨日近视假设，引入终端储能价值进行配对对照：在相同情景池、核权重及近视执行层下，远视模型较近视多支出 38051 元，未达到预设 0.5% 节费采纳门槛，故**保留近视模型**。

针对问题三，引入日内光伏预报更新，采用**四阶段滚动随机规划与情景模型预测控制**，依据新预报重建剩余时段情景，冻结已执行决策并优化后续购电调整。基准配置采用 0.8 分位购电下界与 5 情景执行层，仅用 0:00 预报的全年费用为 1795.60 万元，在所比较的四种嵌套策略中最低；依次新增 6:00、12:00、18:00 更新，费用分别增加 112.53 万元、减少 25.77 万元和 4.72 万元，表明**增加预报更新并不必然降低费用**，其收益需与调整代价共同权衡。

针对问题四，将固定电价扩展为实时波动电价，基于历史信息进行多尺度电价预测，并构造**负载、光伏与电价的联合残差情景**，保留同期相关性。沿用上述基准配置，日前购电与日内滚动调整策略的全年结算费用分别为 1862.76 万元和 1939.87 万元。价格信息价值对比显示，基准配置下完美价格信息未降低回测费用；取消分位下界并将执行层情景数增至 50 后，两种策略分别节费 14.18 万元和 16.15 万元，说明信息收益与计划保守程度及执行层情景覆盖有关。

关键字： 两阶段随机规划 情景模型预测控制 情景生成 储能调度 价格信息价值

一、 问题重述

1.1 问题背景

微网将分布式光伏、居民用电与储能连接起来，为清洁能源的本地消纳提供了途径。然而，光伏出力与用电需求存在时间错位，外网电价也有峰谷差异。因此，非孤岛式微网需要联合安排外网购电与储能充放电，在保障供电的同时降低购电费用，发挥储能的经济价值。

1.2 问题要求

依据典型日数据、全年负载与光伏实测数据、分时光伏预报及全年电价，结合储能参数，需解决以下问题：

问题一：在给定光伏预测及每日电价、负载相同的条件下，于 0:00 制定全天购电与充放电方案，满足负载、设备限制及日初日末储电量相等的要求，并计算购电费用。

问题二：在电价固定、负载与光伏随时间变化的条件下，于每天 0:00 制定购电计划；供电不足时按交易电价的 5 倍紧急购电，其余费用按计划量结算，求取相应购电与储能策略。

问题三：利用每天 0:00、6:00、12:00 和 18:00 发布的未来 24 小时光伏预报，制定并调整购电计划；下调部分按交易电价的 50% 支付违约金，上调部分按 1.5 倍计价，综合计划、调整与紧急购电费用，分析日内修正的必要性。

问题四：将固定电价扩展为实时波动电价，重新计算问题二、三，比较不同信息条件与调整方式下的购电、储能策略及运行费用。

二、 问题分析

2.1 问题一的分析

首先将一天划分为 144 个 10 分钟时段，把负载与光伏功率换算为时段电量。随后，以购电量、充放电量及储电量为变量，建立全天购电费用最小的线性规划，将供需平衡、储能递推、容量与功率限制及日循环条件纳入约束。求解后汇总指定时段的购电与充放电方案，核验能量平衡和设备边界，并与无储能方案比较购电费用。

2.2 问题二的分析

问题二按情景构造、计划决策与实际曲线回测三步展开。先依据月度差异、周周期及近期供需特征，筛选同季节历史完整日曲线；通过核加权赋予情景概率，以能量分数确定带宽。再建立两阶段随机线性规划：第一阶段确定各情景共用的日前购电量，第二阶段安排充放电、紧急购电与剩余电量弃置，最小化期望日费用。最后固定购电计划，在实测曲线上求解补救方案，将日末储电量传至次日，逐日计算费用；另对赋权方案和跨日终端价值作对照检验。

2.3 问题三的分析

首先利用历史数据预测负载，将整点光伏预报经保形插值与区间积分转为 10 分钟电量，再叠加同季节历史日的联合残差构造情景。在 0:00 求解条件两阶段随机规划，得到原计划；6:00、12:00 和 18:00 到来时，冻结已执行时段，依据新预报和当前储电量重建情景，综合计划、调整及紧急购电费用与跨日终端价值，优化剩余计划，形成四阶段滚动策略。执行层采用情景模型预测控制（MPC），每 10 分钟重算并只执行当前动作。最后比较不同预报更新组合的实际结算费用，判断各时刻修正计划的必要性。

2.4 问题四的分析

先利用已知历史价格，按日内、周内和季节三个尺度构建电价预测，再抽取同一历史日的负载、光伏与电价残差，生成联合情景。随后将情景电价纳入购电目标与储能控制，分别求解日内不修改计划、按预报滚动修正计划两种方案，完成问题二、三的波动电价重算。最后按实际电价结算，比较两种策略的费用，并设置提前已知全天价格的对照，评价价格预测与信息更新的经济价值。

四个问题层层递进：问题一在确定条件下建立购电与储能的联合调度基线；问题二引入负载与光伏的不确定性，建立两阶段随机规划；问题三在此基础上加入日内预报更新，形成四阶段滚动与情景 MPC；问题四再把电价由固定扩展为实时波动。四个问题共享同一套能量平衡与储能递推约束，仅改变不确定性来源与决策时点，从而保证结果横向可比。

三、 模型假设

为突出购电与储能协同决策的主要矛盾，在题设条件外作如下假设：

1. 假设以 10 分钟区间平均功率表示供需，购电及充放电量可连续调节，忽略设备启停时间与最小动作幅度。
2. 假设储能充放电效率保持不变，除充放电损耗外，忽略线路损耗、自放电、寿命衰减及运维费用。
3. 假设外网购电不另设功率上限，微网不向外网售电；无法利用的剩余电量允许弃置，无额外处置费用。
4. 假设同季节、运行特征相近的历史日可近似反映待决策日的条件分布；情景构造仅使用决策时已知信息。
5. 假设紧急购电仅补足当前负载缺口，不用于储能充电；问题三、四可在当前区间内依据已观测供需及时调节。
6. 问题三及其波动电价扩展中，假设按最终生效购电量相对 0:00 原计划结算一次调整费用，已执行时段不追溯修改；逐次结算另作敏感性分析。

四、 符号说明

符号	意义	符号	意义
$t, \mathcal{T}$	十分钟时段索引及集合; $t = 1, \dots, 144$	$d, \mathcal{D}$	日期索引及数据日期集; 2025 年的 365 个日期
Δ	单时段长度; 1/6	$\mathcal{D}_{\text{out}}$	输出日期集; 2025-02-01 至 12-31, 共 334 个日期
P_t	问题一至三固定购电价格	k	预报发布阶段; 0、1、2、3 对应 0、6、12、18 时
ℓ_t	典型日负载功率	ω, Ω_d	情景索引及目标日情景集合 (按日构造)
P_t^{PV}	典型日光伏预测功率	$\mathcal{H}_d$	目标日的候选历史日集合
$L_t, L_{d,t}$	典型日、实际逐日负载电量	$\pi_{d,\omega},$ $\pi_{d,k,\omega}$	日计划、更新阶段的情景权 重; 非负且和为 1
$R_t, R_{d,t}$	典型日、实际逐日可用光伏 电量	$L_{d,t}^\omega,$ $L_{d,k,t}^\omega$	日计划、更新阶段的负载情 景电量
N_t	典型日净负荷电量; $L_t - R_t$	$R_{d,t}^\omega,$ $R_{d,k,t}^\omega$	日计划、更新阶段的光伏情 景电量
$G_t, G_{d,t}$	典型日、逐日计划购电量	$N_{d,k,t}^\omega$	更新阶段情景净负荷电量; 情景负载减光伏
$C_t, C_{d,t}^\omega$	交流母线侧充电量及其情景 值	$\hat{L}_{d,k,t}$	阶段 k 对时段 t 的负载电量 预测
$D_t, D_{d,t}^\omega$	交流母线侧放电量及其情景 值	$\hat{R}_{d,k,t}$	阶段 k 对时段 t 的光伏电量 预测
$E_t, E_{d,t}^\omega$	紧急购电量及其情景值; 按 交易价 5 倍结算	$G_{d,t}^c$	计划购电中划给储能充电的 份额
$S_t, S_{d,t}^\omega$	时段开始时储电量及其情景 值	$\Delta_{d,t}$	情景加权的正净负荷缺口
W_t^{PV}	纯光伏弃光电量	$G_{d,t}^{(0)}$	0:00 制定的原计划购电量
$W_t, W_{d,t}^\omega$	合并弃置电量, 含弃光与已 购未利用	$G_{d,t}^{(k)}$	阶段 k 更新后的总购电量, 非增量
$\mathcal{T}^+$	储能状态端点集; $\{1, \dots, 145\}$	$G_{d,t}^F$	交割前最终生效的购电量
$S^{\min}, S^{\max}$	储电量安全下限、上限; 1200、10800	$A_{d,t}^{(k)}$	阶段 k 计划相对原计划的绝 对调整量
$\bar{e}$	单时段最大充电或放电量; 约 833.33	$\theta_{d,k}^\omega$	情景日末储能的未来费用代 理
η, η_c, η_d	单程充、放电效率; 主口径 均为 0.9	$P_{d,t}$	问题四实际波动电价
J	问题一全天购电费用	$\hat{P}_{d,t}$	电价点预测
C_{official}	输出期官方结算费用	$P_{d,t}^\omega$	情景电价
$m_{d,t}^p$	电价周同期基线	$\bar{P}_{d,t}^{\text{seas}}$	同季节历史日平均电价
$\gamma_{d,t}^p$	同星期日内价格时段效应, 零和	$\delta_{d,r}^p$	实时位置 r 的电价水平修正
$\varepsilon_{j,t}^p$	历史日电价预测残差		

五、 数据处理和探索性分析

5.1 数据处理

经检查，四个附件的有效数值字段未发现缺失值、非有限值或负功率，电价均为正值，日期与时段的对应关系完整。因此，数据无需进行缺失值插补、异常值修正等额外预处理，可直接用于后续分析。

5.2 探索性分析

微网购电既要满足负载，又要兼顾光伏消纳和运行经济性。围绕这三者的关系，我们从日内供需、月度变化、负载周期以及预报与电价变化四个方面展开分析。

5.2.1 日内供需匹配与储能调度空间

考虑到光伏发电依赖日照，而居民用电遵循生活作息，两者在时间上未必一致。相关资料也指出，太阳能发电充裕的时刻并不总是用电需求最大的时刻，储能的重要作用之一正是缓解这种时间错位。^[1] 因此，我们将附件 1 的负载、光伏预测功率及电价放在同一日内尺度下比较，并以负载电量减去光伏电量定义净负荷，绘得图 1。

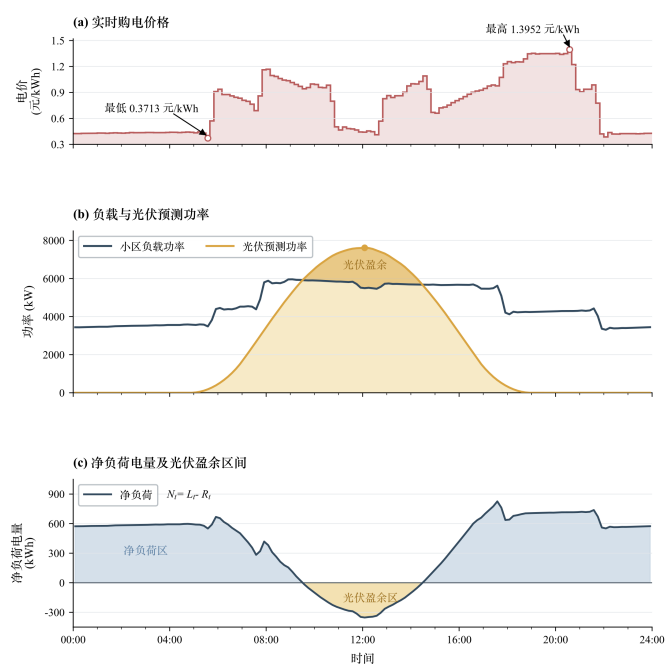


图 1 典型日电价、负载与光伏及净负荷特征

观察图 1，我们发现午间光伏不仅能够覆盖负载，还形成了一段净负荷为负的富余区间；傍晚光伏逐渐退出后，微网又需要较多外部供电。与此同时，凌晨和深夜电价较低，晚间电价较高。这说明微网面临的并非单纯的电量不足，而是发电、用电和购电价格在时间上的不匹配：午间有电可用，却未必能在当时全部消纳；晚间需要供电，又可能恰逢较高电价。

由此，储能的作用可以从两个角度理解。一是将午间富余光伏留待后续使用，提高清洁能源的本地消纳水平；二是利用低价时段充电，在高价时段减少外网购电，为改善

经济性创造空间。两种作用不一定对应同一充放电安排，因此调度时不能只追随负载，也不能只追随电价，而应联合考虑全天供需与储能状态。

进一步比较后，我们还发现附件 1 的负载和光伏曲线与附件 2 的全年同刻均值几乎重合，说明典型日能够反映平均意义下的供需形状。不过，平均曲线也可能掩盖实际日期之间的差异，这促使我们继续考察全年数据。

5.2.2 供需关系的月度变化

查阅有关居民用电的资料可知，气温变化会通过制冷、采暖等需求影响用电；光伏出力则受日照和天气条件影响。^{[2] [1]} 考虑到两者对季节变化的响应未必同步，我们对附件 2 按月汇总：先计算每日累计电量，再求各月的日均值，以避免月份天数不同带来的影响；同时选取 3、6、9、12 月，比较各月的平均日内净负荷，结果如图 2 所示。

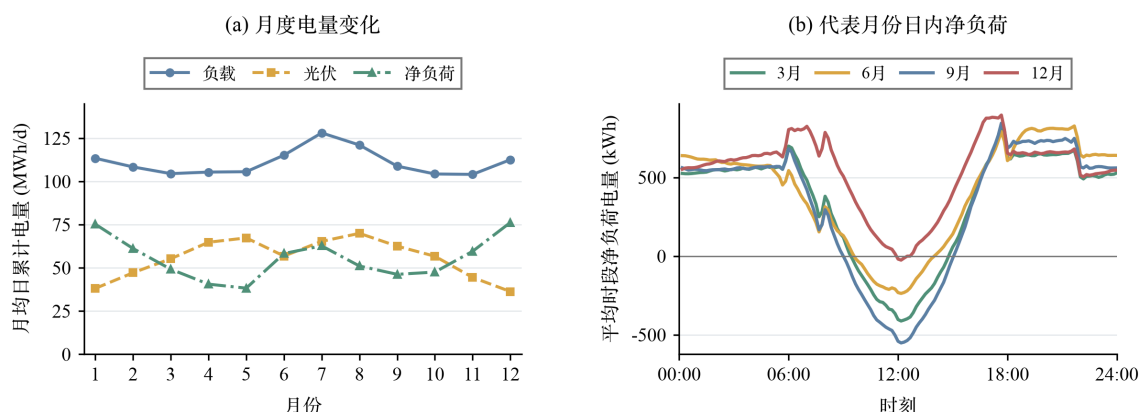


图 2 月均日累计电量与代表月份的平均时段净负荷

从图 2 可以看出，负载与光伏并不是同步升降的。夏季负载虽有所增加，较充裕的光伏也能抵消部分供电需求；冬季光伏贡献减弱，净负荷则明显高于春末。这与居民用能和太阳能资源可能随季节变化的一般认识相符，意味着微网对外部供电的依赖取决于发电与用电的共同变化，不能仅根据负载大小判断。

我们进一步发现，不同月份的净负荷并非只存在总量差异，午间低谷的深度也明显不同：部分月份能够形成较充裕的光伏富余，冬季午间则接近供需平衡。这意味着，一套适合光伏充裕月份的储能安排，未必适合光伏不足的月份；后续决策既要关注当天需要多少电，也要关注这些电量在什么时刻出现。因此，历史样本的选择应考虑月份和季节相近性，而不宜将全年日期不加区分地混合使用。这里识别的是一年样本中的月度特征，尚不能据此确认稳定的年度周期。

5.2.3 居民负载的周周期

除天气因素外，考虑到工作日与休息日的交替可能改变居民在家时间和用电安排，负载还可能存在周尺度上的重复性。为检验这一点，我们比较相隔 1–14 天的同刻负载。由于每天共有的昼夜形状也会使相关性偏高，先减去各时段的全年平均负载，再计算日滞后相关系数，绘得图 3。

我们发现，相关性在相隔 7 天和 14 天处形成明显峰值，且高于邻近滞后，说明负载在日际变化之外保留了较强的周周期。这与以周为单位的工作和生活节奏相吻合，也

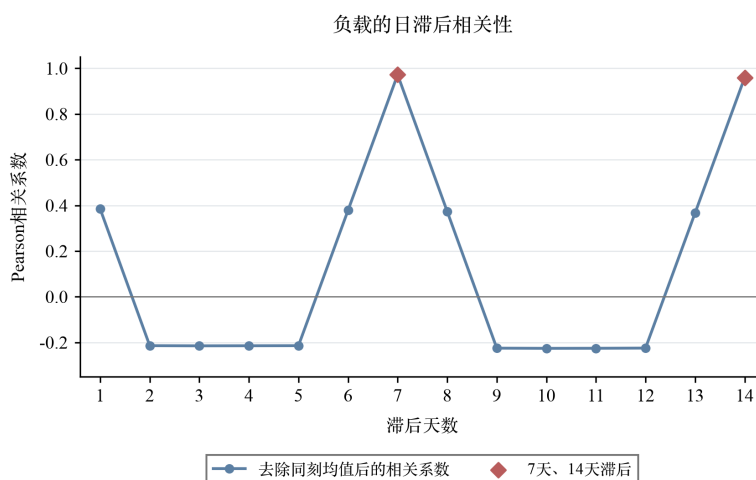


图3 去除同刻均值后的负载日滞后相关性

提示我们：距离最近的历史日期未必最有参考价值，上周同刻的数据反而可能更接近当天的用电模式。

5.2.4 光伏预报不确定性与电价时变性

此前的分析揭示了供需变化中的规律，但购电计划面向未来，预报能否准确反映未来的光伏出力，仍需进一步考察。另一方面，即使供需相近，价格变化也可能改变储能的经济价值。因此，我们将附件3 预报与附件2 同一目标整点的实际光伏功率对齐，在有实际观测的样本中计算不同提前时间的平均绝对误差（MAE）；同时将附件4 按日期与日内时刻排列，直接绘出原始电价热力图，合并展示于图4。

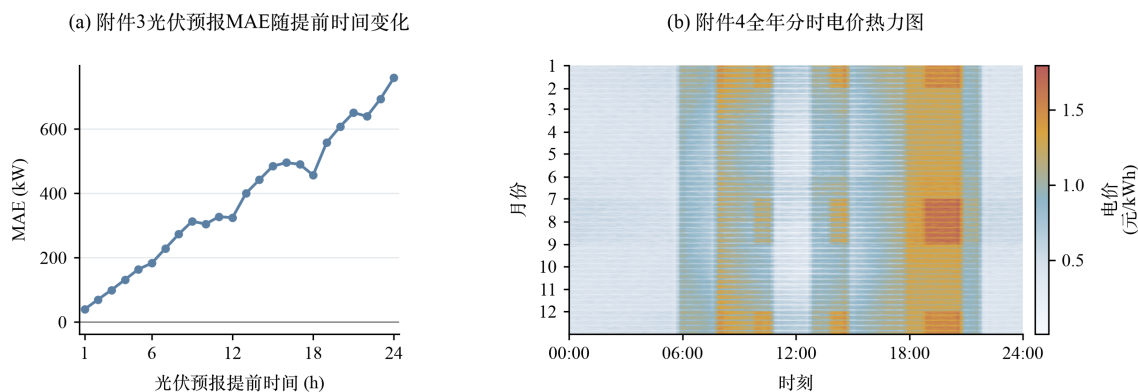


图4 光伏预报误差随提前时间变化与全年分时电价热力图

观察图4(a)，我们发现预报误差随提前时间总体增大，近时刻预报较准确，远时刻偏差则更明显。这与已有研究中较长预测时距通常伴随较低预测精度的结论一致。^[3] 为避免夜间大量近零值掩盖发电时段的偏差，图中仅统计预报值或实际值超过 50 kW 的样本。

从决策角度看，这说明日前预报不宜被当作确定的发电量。随着时间推进，更新预报可能为修正供需判断提供更有价值的信息，微网也有必要在初始计划之外保留调整空间。不过，预报更准确不等于调整一定更划算：减少紧急购电的收益还需与调整费用权

衡，这正是后续研究滚动购电策略的关键。

图 4(b) 则表明，价格波动中仍保留了较清晰的日内峰谷结构，但同一时段的价格水平又随日期变化。这与电价通常受用电需求和发电成本等因素共同影响的背景相符。^[4] 我们据此认为，波动电价并未消除低价充电、高价放电的机会，却使固定时刻、固定电量的调度方式缺乏足够适应性。储能的价值不仅取决于能转移多少电量，还取决于何时转移、对应价差是否足以覆盖充放电损耗。

综合上述分析，我们发现光伏发电与居民用电在时间上存在错位，供需关系又随月份和生活节奏变化，预报偏差与电价波动进一步增加了调度的复杂性。这说明，微网的运行效益不仅取决于能够获得多少电能，更取决于能否在合适的时刻合理配置和利用电能。因此，研究光伏消纳、储能充放电与外网购电的协同优化，对于协调分布式发电与用电需求、保障小区供电并降低购电成本具有实际意义，也体现了微网在清洁能源本地利用与经济运行之间寻求平衡的必要性。上述分析也为后续刻画供需变化、预报不确定性与电价影响，建立购电与储能联合优化模型提供了数据依据。

六、 问题一模型的建立与求解

问题一要求根据附件 1 的电价、负载和光伏预测数据，在每天 0:00 制定间隔 10 min 的全天购电计划，并保持日初与日末储电量一致。结合前文识别的供需与电价错位，本文以购电费用最小为目标，建立考虑能量平衡、储能效率及运行边界的确定性线性规划模型，联合确定购电与充放电策略。

6.1 模型建立

设一天划分为 144 个等长时段，时段长度为 $\Delta = 1/6 \text{ h}$ 。令 P_t 、 L_t 、 R_t 分别表示第 t 个时段的外网电价、负载电量和可用光伏电量；令 G_t 、 C_t 、 D_t 、 W_t 分别表示该时段的外网购电量、储能充电量、储能放电量和弃光量；令 S_t 表示第 t 个时段开始时的储电量。储能容量下限和上限分别为 1200 kWh 和 10800 kWh，单时段最大充放电量为

$$\bar{e} = 5000\Delta \approx 833.33 \text{ kWh}. \quad (1)$$

本文以全天购电费用最小为目标，建立如下目标函数：

$$\min J = \sum_{t=1}^{144} P_t G_t. \quad (2)$$

对任意时段，外网购电、光伏发电和储能放电共同构成供给，负载、储能充电和弃光构成去向，因此有能量平衡约束

$$G_t + R_t + D_t = L_t + C_t + W_t, \quad t = 1, \dots, 144. \quad (3)$$

其中，弃光量只可能来自当期可用光伏，故满足

$$0 \leq W_t \leq R_t, \quad t = 1, \dots, 144. \quad (4)$$

储能充、放电效率均取 $\eta = 0.9$ 。由于 C_t 与 D_t 均按交流母线侧电量计量，储能状态递推关系为

$$S_{t+1} = S_t + \eta C_t - \frac{D_t}{\eta}, \quad t = 1, \dots, 144. \quad (5)$$

储能设备在运行过程中不能过充或过放，同时充放电功率不得超过设备上限，因此有

$$1200 \leq S_t \leq 10800, \quad t = 1, \dots, 145, \quad (6)$$

$$0 \leq C_t, D_t \leq \bar{e}, \quad t = 1, \dots, 144. \quad (7)$$

题目要求 0:00 与 24:00 储电量相同。该约束可以防止模型通过消耗初始储能来虚假降低购电费用，即

$$S_{145} = S_1. \quad (8)$$

此外，外网购电量非负：

$$G_t \geq 0, \quad t = 1, \dots, 144. \quad (9)$$

综上，问题一可归结为以下线性规划模型：

$$\begin{aligned} \min \quad & J = \sum_{t=1}^{144} P_t G_t, \\ \text{s.t.} \quad & G_t + R_t + D_t = L_t + C_t + W_t, \quad t = 1, \dots, 144, \\ & S_{t+1} = S_t + \eta C_t - \frac{D_t}{\eta}, \quad t = 1, \dots, 144, \\ & 1200 \leq S_t \leq 10800, \quad t = 1, \dots, 145, \\ & 0 \leq C_t, D_t \leq \bar{e}, \quad t = 1, \dots, 144, \\ & 0 \leq W_t \leq R_t, \quad G_t \geq 0, \quad t = 1, \dots, 144, \\ & S_{145} = S_1. \end{aligned} \quad (10)$$

由于外网电价为正、储能存在充放电损耗，且微网不向外网售电，同一时段同时充电和放电不会带来费用优势，反而对应无效的能量循环。因此，本文不额外引入充放电互斥二元变量，而是在求解结果中检查是否存在同时充放电现象。

6.2 模型求解

将决策变量按

$$\mathbf{x} = (G_1, \dots, G_{144}, C_1, \dots, C_{144}, D_1, \dots, D_{144}, W_1, \dots, W_{144}, S_1, \dots, S_{145})^\top \quad (11)$$

排列后，上述模型可写成标准线性规划形式：

$$\begin{aligned} \min \quad & \mathbf{c}^T \mathbf{x}, \\ \text{s.t.} \quad & \mathbf{A}_{\text{eq}} \mathbf{x} = \mathbf{b}_{\text{eq}}, \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u}. \end{aligned} \quad (12)$$

模型共含 721 个连续变量和 289 个等式约束，目标函数与全部约束均为线性，可行域为多面体，属于可直接精确求解的连续线性规划。因此采用 SciPy 的线性规划求解器 `scipy.optimize.linprog`（HiGHS 方法）求解，得到最小购电费用

$$J^* = 35118.60 \text{ 元}. \quad (13)$$

附件 1 未指定典型日的日初储电量。辅助优化得到等价最优区间 $S_1 \in [8550, 8700]$ kWh。以 $J(s)$ 表示固定 $S_1 = S_{145} = s$ 时的最小购电费用，图 5 展示其相对最优值的增量：区间内费用不变，区间外随初始储电量偏离而增加。因此，本文取 $S_1 = 8550$ kWh 作为展示和填表的代表方案，该值并非唯一最优初始状态。

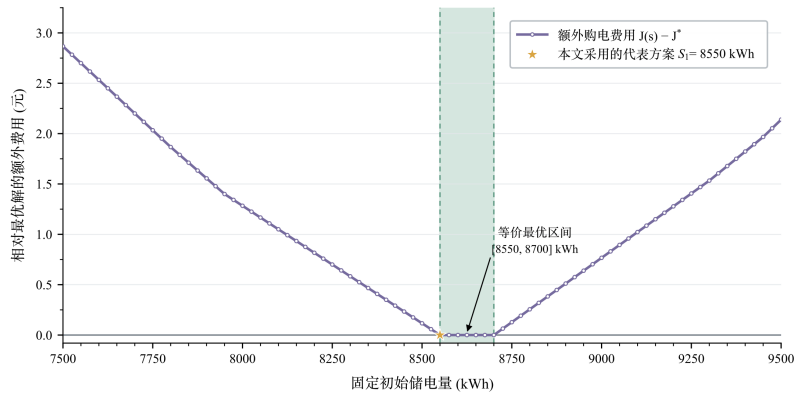


图 5 初始储电量对最优购电费用的局部影响

6.3 结果分析

题目指定的购电结果见表 1，最优方案全天购电 59482.70 kWh。

表 1 问题一指定时段购电量及全天结果

时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量/kWh		59482.70	全天购电费/元		35118.60

分时段充放电结果见表 2，调度轨迹见图 6。低价时段及午间光伏富余时，储能补充电量；早晚高价阶段则通过放电减少外网购电。日初与日末储电量均为 8550.00 kWh，运行范围为 1200 ~ 10800 kWh，且逐时段检查未发现同时充放电现象。可见，最优策略由电价、光伏出力和储能状态共同决定，不能仅按当期负载缺口安排购电。

表 2 问题一储能设备分时段充放电量及日初日末储电量

时间段	充电量/kWh	放电量/kWh	时间段	充电量/kWh	放电量/kWh
00:00–04:00	1666.67	0.00	04:00–08:00	833.33	6365.84
08:00–12:00	4787.96	1703.00	12:00–16:00	5286.04	91.10
16:00–20:00	0.00	5780.13	20:00–24:00	8166.67	2859.87
0:00 储电量/kWh	8550.00		24:00 储电量/kWh	8550.00	

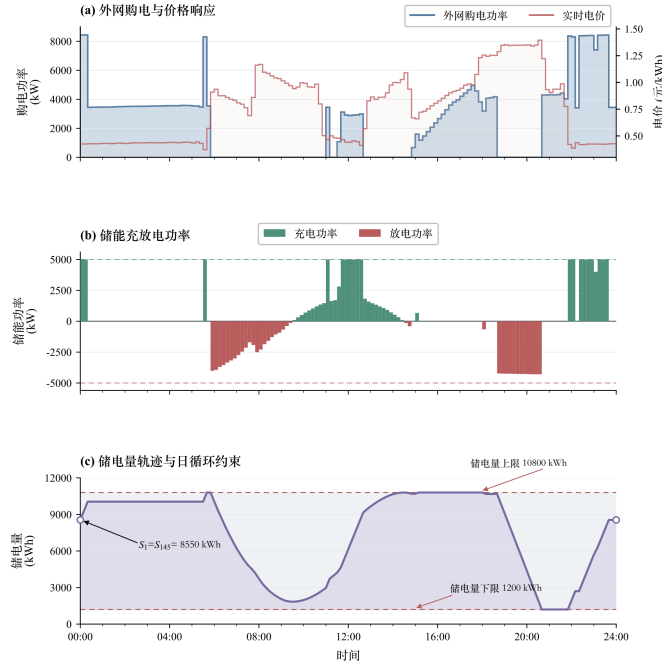


图 6 最优购电、储能充放电与储电量轨迹

为评价储能调度的经济效果，构造无储能基准方案：各时段优先利用光伏满足负载，不足部分由外网购电，富余光伏直接弃置。此时

$$G_t^0 = \max\{L_t - R_t, 0\}, \quad J_0 = \sum_{t=1}^{144} P_t G_t^0. \quad (14)$$

表 3 显示，储能优化节省购电费用 12933.45 元，节省率为 26.92%，并将全天弃光量降为 0。图 7 进一步表明，节费主要来自早晚高价阶段的放电，提前充电和日末补能则形成部分时段的额外支出。因此，储能调度应评价全天净收益：低价补能成本可由后续放电收益抵消，同时富余光伏也得以在本地消纳。

表 3 问题一储能优化的经济效果

指标	无储能基准	储能优化	变化量
全天购电量/kWh	61789.94	59482.70	−2307.24
全天弃光量/kWh	6247.96	0.00	−6247.96
全天购电费用/元	48052.05	35118.60	−12933.45
购电费用节省率	—	—	26.92%

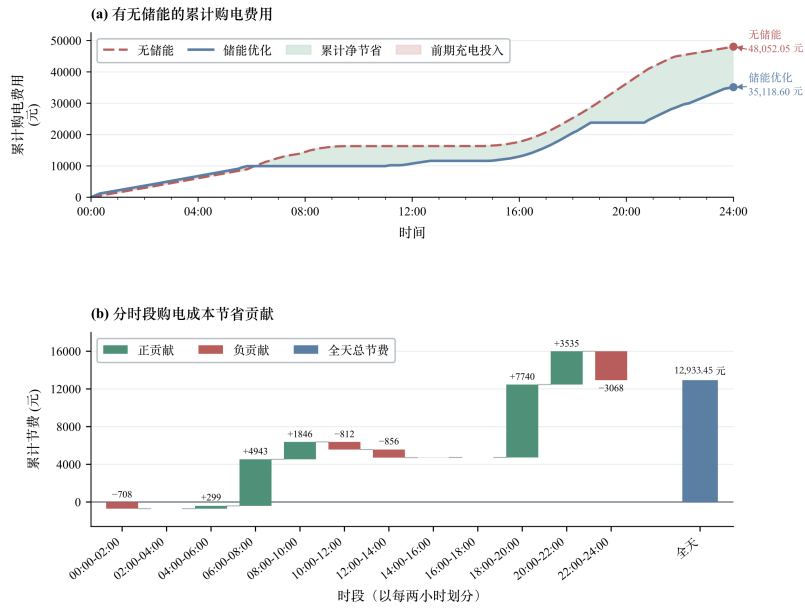


图 7 有无储能的累计购电费用及分时段节费贡献

七、 问题二模型的建立与求解

问题二在问题一的基础上放松了确定性假设。每天的电价曲线相同且完全已知，不确定性全部来自负载与光伏功率的逐时段变化。微网向外的供电不得低于小区负载，一旦不足，缺口部分必须以交易时刻电价的 5 倍紧急购电；除紧急购电费用以外，其余时段的购电费用一律按计划购电量结算，与计划电量是否被实际使用无关。

问题一假设电价与负载逐时段已知，唯一的不确定来源是光伏预测偏差，模型可退化为确定性的单日线性规划；问题二把不确定性整体搬到负载侧与光伏侧，0:00 制定计划时当天的负载曲线与光伏曲线尚未揭晓，决策只能依据此前的历史数据，不得引入任何前视信息。微网向外的供电不得低于小区负载，计划不足的部分必须以 5 倍电价紧急购入，计划偏多的部分只能存入储能或弃置，两类偏差的代价明显不对称。单看一个时段，这一取舍具有**报童问题**的结构，但储能状态在日内 144 个时段之间连续递推，需要刻画的并非各时段彼此独立的不确定性，而是整条净负荷曲线的联合变化，因此本文把问题形式化为带储能跨时段耦合的**两阶段随机线性规划**：计划购电量在 0:00 一次确定，对所有可能的次日情形取同一值；储能充放电、紧急购电与弃置电量则随当天揭晓的实际曲线调整。问题的难点在于次日净负荷的联合分布未知，只能由有限的历史数据推断，本文以历史日的完整曲线作为情景、以经验期望近似真实期望，情景的选取范围、采样单位与前视约束则一并由数据自身的月度差异、周周期与残差相关性决定。

7.1 数据预处理与探索性分析

7.1.1 结构性证据

情景池的构造依据来自四方面的数据事实。其一是**月度差异**：负载与光伏的月度变化不同步，净负荷起伏更明显，5 月与 12 月的月均日累计净负荷分别为 38.28 MWh/d 与 76.26 MWh/d，后者约为前者的 1.99 倍，代表月份的日内净负荷都呈午间下降、傍晚

回升，午间低谷的深度差别很大，全年样本不宜混合，应优先取与决策日季节相近的日期。其二是负载的**周周期**：先减去各时段的全年均值，再算相隔 1 至 14 天的同刻相关系数，滞后 7 天与 14 天分别达到 0.9722 与 0.9600，明显高于邻近滞后（见图 3），说明上周同一时段的观测更可能保留相似规律。其三是预测误差的**日内延续性**：以负载上周同刻值与光伏过去 7 日同刻均值构成的组合基线剥离可预测成分后，相邻 10 分钟净负荷残差的相关系数仍有 0.8497，情景因此以完整日曲线为采样单位。**星期几**是第四个来源，可解释去均值后方差的 85.28%；日总电量最低的两天是周五与周六，约 78.3 与 78.4 MWh，其余五天约 123.9 至 124.1 MWh，相差约 1.58 倍，这否定了把周六与周日同划为周末的假设，因为周日是高负载日。

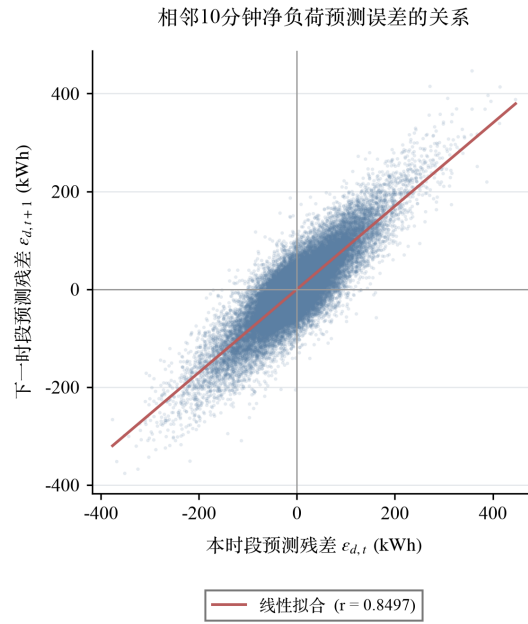


图 8 相邻 10 分钟净负荷预测误差的关系

图 8 显示相邻时段的净负荷预测误差之间存在明显的正相关：上一时段偏高的误差倾向于延续到下一时段，与相邻 10 分钟残差相关系数 0.8497 的定量结果一致。这意味着净负荷的不可解释成分并非逐时段独立，若在情景生成时对时段独立抽样，将丢失这一日内惯性，从而低估整条净负荷曲线的波动幅度；因此本文以完整历史日曲线作为情景单位，逐日整段抽取，保留时序相关性。

7.1.2 特征集合与核权重

最终处境由季节圆周位置、上周同日净负荷、前三日光伏均值、上周同日负载、前七日净负荷均值与低负载日标志构成，特征集合为 `season`、`net_lag_7d`、`pv_mean_3d`、`load_lag_7d`、`net_mean_7d` 与 `low_load_day`。以 z_d 与 z_w 表示验证日与情景日的标准化处境向量，核权重取高斯形式

$$\pi_{d,\omega} = \frac{\exp[-\|z_d - z_\omega\|_2^2 / (2h^2)]}{\sum_{j \in \Omega_d} \exp[-\|z_d - z_j\|_2^2 / (2h^2)]}, \quad (15)$$

其中 $\|z_d - z_\omega\|_2$ 是标准化处境距离, h 为带宽, 控制权重衰减的快慢。带宽以平均能量分数唯一选优, 冻结取值为 $h = 1.043841$ 。评分同样采用平均能量分数, 以 x_ω 与 y_d 表示两日的 144 维净负荷曲线, 定义

$$ES_d = \sum_{\omega \in \Omega_d} \pi_{d,\omega} \|x_\omega - y_d\|_2 - \frac{1}{2} \sum_{\omega, \omega' \in \Omega_d} \pi_{d,\omega} \pi_{d,\omega'} \|x_\omega - x_{\omega'}\|_2, \quad (16)$$

其中第一项度量情景曲线到真值的加权距离, 第二项是情景间的加权分散度并从分数中扣除; 分数越小越好。特征筛选与带宽搜索的完整过程详见附录 D。

7.1.3 赋权方案的采用依据

全年 334 天的平均能量分数为: 等权基线 1334.486 kW, 旧特征集 1248.040 kW, 修订特征集 551.071 kW。修订方案相对等权基线改善 58.706%, 改善日数 328/334; 相对旧特征集改善 55.845%, 改善日数 318/334, 两者均超过预设的 3% 采纳门槛。核加权同时提高了权重集中程度, 有效情景数 K_{eff} 的均值为 12.17, 范围为 1.01 至 40.08。为排除评分口径造成的假象, 另做决策层抽样回测兜底: 每月取前 2 天共 22 天, 三套权重分别求解决策层与执行层并在真实曲线上结算, 修订特征集的总费用为 1,125,932.8 元, 比等权基线低 10.354%。

7.2 模型建立

7.2.1 模型假设

1. **跨日近视:** 储能的跨日影响有限, 即将储能电量留到第二日消耗的边际回报处处不超过今日释放的即时收益。该假设的口头论证 (见附录) 在当前模型族内不可检验, 本工作把它改写为待检验命题, 并在同一情景池、同一核权重、同一执行层下用含终端价值的远视模型做配对对照, 检验设计与结论见后文。
2. **已购电量可以弃置:** 计划购电按量结算、无需用尽, 允许 $W_{d,t}^\omega > 0$ 。
3. **紧急购电不进储能:** 紧急购电只用于补足负载缺口, 不作为储能充电的来源。该性质不能由 $5P_t > P_t$ 自动保证, 因为跨时段被比较的是低价时段的 $5P_t$ 与高价时段的 $5P_{t'}$, 储能让两者之间的套利成立; 又因充电量是情景特定变量而计划购电量是全情景共享变量, 模型有动机用情景内的廉价紧急电充电替代提高共享计划。本模型因此把它写成显式的线性约束, 即形式化建模中的充电来源约束。
4. **购电无功率上限:** 题面未对计划购电与紧急购电设置功率限制, 模型不设。
5. **情景代表性:** 次日与 $\mathcal{H}_d$ 中同季节历史日近似可交换, Ω_d 的经验分布可作为次日净负荷条件分布的近似。

7.2.2 形式化建模

情景生成。 对每个 $d \in \mathcal{D}$, 在 0:00 从候选历史池 $\mathcal{H}_d$ 中以整天为单位抽取情景并按数据预处理一节得到的核函数赋权, 得到 Ω_d 与 $\pi_{d,\omega}$ 。每个情景 ω 直接提供历史日 ω 的完整实测曲线。由于 $\mathcal{H}_d$ 只含 $j < d$ 的历史日, 情景集合及其权重不含任何未来信息。整日抽取的好处是历史日天然保存了日内的时序结构, 不需要额外假定残差的分布形式。

决策变量。第一阶段变量为计划购电量 $G_{d,t} \geq 0$ 与其中划给储能充电的份额 $G_{d,t}^c$ ，二者在 0:00 一次性确定并对所有情景取同一值；第二阶段变量为逐情景的储能充放电量、储电量、紧急购电量与弃置电量，在情形揭晓后确定。

目标函数。计划购电量在 0:00 定死，紧急购电量随次日实际曲线浮动，模型要在确定发生的计划电费与期望的紧急购电费之间取舍：

$$\min \sum_{t \in \mathcal{T}} P_t G_{d,t} + \sum_{\omega \in \Omega_d} \pi_{d,\omega} \sum_{t \in \mathcal{T}} 5 P_t E_{d,t}^\omega + \varepsilon \sum_{\omega \in \Omega_d} \pi_{d,\omega} \sum_{t \in \mathcal{T}} (C_{d,t}^\omega + D_{d,t}^\omega). \quad (17)$$

其中 $P_t G_{d,t}$ 是按附件 1 电价结算且确定发生的计划购电费； $5 P_t E_{d,t}^\omega$ 是情景 ω 下的紧急购电费，单位缺货成本相当于 $4 P_t$ ；权重归一，第二项即次日紧急购电费的期望估计； $\varepsilon = 10^{-6}$ 元/kWh 是去退化项，只用于在等价的充放电路径中挑出动作最少的一条，不改变计划购电量与主成本。目标函数只覆盖当天，不含日末储能对次日的价值，这是跨日近视假设在模型中的落点。两项系数的差异决定了计划的保守程度：过剩只付 P_t ，缺货却要付 $5 P_t$ ，因此把计划量从均值向上推到净负荷分布的 0.8 分位数是有利的；储能项在此的作用是把低价时段的富余电量搬到高价时段，它自身不改变这一分位数结构的定性结论。第二项把 144 维的不确定性压缩为一个标量，使第一阶段决策可以在不知道具体情形的前提下作出。

能量平衡约束。每个时段的电能来源与去向必须相等：

$$G_{d,t} + E_{d,t}^\omega + R_{d,t}^\omega + D_{d,t}^\omega = L_{d,t}^\omega + C_{d,t}^\omega + W_{d,t}^\omega, \quad \forall \omega \in \Omega_d, t \in \mathcal{T}. \quad (18)$$

其中 $G_{d,t}$ 与 $E_{d,t}^\omega$ 是外购电量， $R_{d,t}^\omega$ 是情景光伏， $D_{d,t}^\omega$ 是储能放电； $L_{d,t}^\omega$ 是情景负载， $C_{d,t}^\omega$ 是储能充电， $W_{d,t}^\omega$ 是弃置电量，弃光与已购未利用合并计入后者。弃置变量显式出现在去向一侧，使计划购电量可以多于当日实际所需，模型不必把多余的计划电强行塞进储能。该约束对每个情景逐时段成立，是后续全部约束的共同基础。等式两侧都是原始物理量，未经任何线性化处理。

储能递推与共同出发点。储能状态在时段之间递推，充电按 η_c 折算入库，放电按 η_d 折算出库：

$$S_{d,t+1}^\omega = S_{d,t}^\omega + \eta_c C_{d,t}^\omega - \frac{D_{d,t}^\omega}{\eta_d}, \quad S_{d,1}^\omega = S_{d,1}, \quad \forall \omega \in \Omega_d, t \in \mathcal{T}. \quad (19)$$

其中 $S_{d,t}^\omega$ 是情景 ω 下时段 t 开始时的储电量， $\eta_c = \eta_d = 0.9$ 使单程损耗为 10%、往返损耗为 19%。第二式是共同出发点约束，所有情景从 0:00 实测的同一储能状态出发。它的作用是把情景差异限制在负载与光伏两类外生曲线上：若允许各情景自由选择起始储电量，模型就能用起点差异掩盖计划购电量的不足，非预期性约束会被架空。递推式在情景之间彼此独立，唯一的耦合来自第一阶段变量与共同出发点。

储能边界、充放电功率与非负性。储电量、充放电动作与购弃电量都必须落在设备与题设允许的范围内：

$$S_{d,t}^{\min} \leq S_{d,t}^\omega \leq S_{d,t}^{\max}, \quad \forall t \in \mathcal{T}^+; \quad 0 \leq C_{d,t}^\omega, D_{d,t}^\omega \leq \bar{e}, \quad \forall t \in \mathcal{T}; \quad G_{d,t}, E_{d,t}^\omega, W_{d,t}^\omega \geq 0, \quad (20)$$

以上约束对所有 $\omega \in \Omega_d$ 成立。其中 $S^{\min} = 1200 \text{ kWh}$ 与 $S^{\max} = 10800 \text{ kWh}$ 是储电量上下限， $\bar{e} \approx 833.33 \text{ kWh}$ 是单时段最大充放电能量。三组界分别防止储能过充过放、限制充放电功率在额定值以内、保证购电量与弃置量非负。储能动作因此只能在 1200 至 10800 kWh 的电量区间与 833.33 kWh 的时段功率预算内开展，后文出现的日末储电量停在下限这一现象并非边界所迫，而是目标函数的选择。

充电来源约束。为防止模型用情景内的紧急购电给储能充电，充电能力必须与全情景共享的第一阶段变量绑定：

$$0 \leq G_{d,t}^c \leq G_{d,t} - \Delta_{d,t}, \quad C_{d,t}^\omega \leq \max(0, R_{d,t}^\omega - L_{d,t}^\omega) + G_{d,t}^c, \quad \forall \omega \in \Omega_d, t \in \mathcal{T}. \quad (21)$$

其中 $G_{d,t}^c$ 是计划购电量中划给储能充电的份额，第一式把它上限取为扣除期望净负荷缺口 $\Delta_{d,t} = \sum_{\omega} \pi_{d,\omega} \max(0, L_{d,t}^\omega - R_{d,t}^\omega)$ 之后的余额，即计划电先留给负载；第二式规定充电只能来自该情景该时段的光伏富余与这份共享份额之和。若不加这两条，模型会出现套利：低价时段的 $5P_t$ 低于高价时段的 $5P_{t'}$ ，用情景内的紧急电充电再到高峰放电替代紧急购电有利可图，全年 365 天中有 103 天出现该行为。这两条是充电侧唯一的来源限制，放电侧不受此限，因为放电只减少储能存量，不引入新的外购电量。

执行层。决策层输出计划购电量之后，白天的实测曲线揭晓，需要按实际数据确定补救动作：

$$\begin{aligned} \min & \sum_{t \in \mathcal{T}} (5P_t E_{d,t} + \varepsilon (C_{d,t} + D_{d,t})) \\ \text{s.t.} & \quad C_{d,t} \leq \max(0, R_{d,t} + G_{d,t} - L_{d,t}), \\ & \quad E_{d,t} \leq \max(0, L_{d,t} - R_{d,t} - G_{d,t}). \end{aligned} \quad (22)$$

其中 $A_{d,t} = R_{d,t} + G_{d,t} - L_{d,t}$ 是扣除负载之后的可用电量，充电上界取其正部，紧急购电上界取其负部。执行层的 $G_{d,t}$ 是已知常数，缺口时段迫使 $C_{d,t} = 0$ ，富余时段迫使 $E_{d,t} = 0$ ，因此不存在一边紧急购电一边充电的时段，该结论精确成立。该互斥性是负载优先的直接推论：先满足负载，再用余量充电，两者不可能同时为正。求解该问题得到实际执行路径，也就是结果文件中充放电量与紧急购电量两张工作表的数值来源，其中 $E_{d,t} > 0$ 的时段与购电量按表 3 格式填报。执行层与决策层的区别在于它使用实测曲线，因此求解结果是唯一的实际路径，不再带有情景维度。

跨日链与全年结算。储能在相邻两日之间链式衔接， $S_{d+1,1} = S_{d,145}$ ，2025-01-01 的起点日初储电量取 6000 kWh；1 月作为冷启动期滚动储能状态但不输出结果。全年官方结算费用为

$$C_{\text{official}} = \sum_{d \in \mathcal{D}_{\text{out}}} \sum_{t \in \mathcal{T}} (P_t G_{d,t} + 5 P_t E_{d,t}), \quad (23)$$

其中 $G_{d,t}$ 与 $E_{d,t}$ 都是实际执行路径上的量，求和范围为输出日期集合与全部 144 个时段。该口径只含真实发生的计划购电费与紧急购电费，是全文比较各方案费用的统一尺度。

7.2.3 跨日近视假设的检验设计

跨日近视直接决定日末储电量与弃置电量的水平，本文把它改写为可判定的命题并按事先固定的口径补做检验，论证史见附录 A 至附录 C。

原口头论证的承重环节是一条充分条件：终端价值函数切线斜率的最大绝对值不超过晚高峰电价的最小值。该条件在当前模型族内不可检验，因为其左端所引用的量只在含终端价值的模型中才有定义，近视模型内没有任何最优性条件涉及它。否证的对象因此不是跨日近视近似不准这一数值判断，而是该论证构成对跨日近视假设的检验这一方法论主张，该结论只依赖模型结构，与具体数据无关。

由于充分条件不可检验，本文补做远视对照：在目标函数中恢复终端价值项，并用分段线性下界从下方支撑终端价值函数，记 θ_d^ω 为情景 ω 下的终端价值辅助变量， $a_{d,q}$ 与 $b_{d,q}$ 为第 q 条切线的斜率与截距，则终端价值以

$$\theta_d^\omega \geq a_{d,q} S_{d,145}^\omega + b_{d,q}, \quad q = 0, \dots, Q_d - 1, \quad \forall \omega \in \Omega_d \quad (24)$$

自下方被支撑。配对对照在同一情景池、同一核权重与逐元素相同的执行层下进行，远视模型的全年官方结算费用为 17,036,556.63 元，比近视基线高 38,051 元，相对 -0.2233% ，未达事先固定的 0.5% 采纳阈值，因此保留近视模型。该负结果只在决策层远视而执行层近视这一具体口径下成立，既不构成对跨日近视假设的证明，也不回答跨日近视究竟有多大代价，完整的配对数据与适用边界见附录 C。

7.3 模型求解

求解按三步推进，三步的先后关系固定。

第一步，决策层。对每一天，用当日候选历史池与冻结的核权重求解两阶段随机线性规划，得到全时段计划购电量 $G_{d,t}$ 。该步输入只含 $j < d$ 的历史日，不含当日及以后的任何观测。

第二步，执行层。白天的实测负载与光伏揭晓后，固定第一步给出的 $G_{d,t}$ ，把实测曲线代入补救问题求解，得到实际执行路径 $C_{d,t}, D_{d,t}, S_{d,t}, E_{d,t}, W_{d,t}$ ，也就是结果文件中充放电量与紧急购电量两张工作表的数值来源。

第三步，跨日链。令 $S_{d+1,1} = S_{d,145}$ 把日末储电量传递到次日，再回到第一步推进下一天，构成滚动时域（rolling horizon）决策链。滚动自 2025-01-01 的 6000 kWh 初始储电量开始，1 月作为冷启动期只推进储能状态、不输出结果，因此 2 月 1 日的日初储电量是滚动结果而非重置值。

7.4 结果分析

7.4.1 模型验证

全年 365 天的求解结果全部通过能量平衡、储能递推、储能边界、充放电功率、充放电互斥、紧急购电与充电互斥、跨日链与费用重算等各项形式化断言，最大残差量级为 10^{-13} 。**无前视泄漏检验通过：**把目标日及其之后的原始观测整体扰动后重建处境特征、候选池与核权重并重跑该日决策，候选池、权重与计划购电量逐元素完全不变，逐日决策输入不含任何未来信息。

一处需要如实记录的模型性质是**弃置电量**。全年弃置电量为 3,848,719.1 kWh，占计划购电量的 17.61%，是全年充电量 624,209.4 kWh 的 6.166 倍，334 天中有 333 天出现弃置，计划购电总量超出实际净负荷 3,911,474.8 kWh。该现象是报童式计划与跨日近视假设共同作用的结果：缺货成本 $4P_t$ 与过剩成本 P_t 使最优计划落在净负荷分布的 0.8 分位数上，多数日子的计划必然高于当日实际净负荷；又因跨日近视，日末储电量每天都在下限，当日用不掉的多余电量既不能留到次日，也不值得为当天并不存在的缺口充电，因为充电只增加往返损耗，于是计入弃置。该弃置在给定模型内是最优的，但它同时标出了跨日近视假设的代价上限，也是检验该假设时最值得复查的一项。该性质的完整复算见附录 A 与附录 C。

7.4.2 敏感性分析

充放电效率口径。题面给出的充放电效率为 90%，这允许两种读法：分别取充电效率与放电效率为 0.9，或取往返效率为 0.9 从而单程效率为 $\sqrt{0.9} \approx 0.9487$ 。本文采用前者，即 $\eta_c = \eta_d = 0.9$ ，单程损耗 10%、往返损耗 19%。若改用后者，往返损耗由 19% 降到 10%，储能的套利与削峰收益同步上升，储能动作的幅度会增大，但计划购电量与弃置电量的相对关系不变。

核加权方案与带宽。三套赋权方案的排序在评分与费用两个口径下一致：平均能量分数分别为 551.071、1248.040 与 1334.486 kW，修订方案相对等权基线改善 58.706%；决策层抽样回测的总费用分别为 1,125,932.8、1,203,114.1 与 1,255,970.2 元，与评分排序相同。带宽在 [0.05, 20] 上经 25 点粗搜与三轮 17 点细搜共 77 个候选唯一选优，得到内部最优 $h = 1.043841$ ，未触及搜索边界，说明结论对赋权方案与带宽的选择敏感。带宽搜索过程与三套方案的全部对比数据详见附录 D。

有效情景数与情景覆盖。有效情景数 K_{eff} 的均值为 12.17，范围为 1.01 至 40.08。权重集中会缩小情景覆盖并可能低估尾部风险，这是核加权方案的固有代价，本文以决策层抽样回测与形式化核验两条独立证据兜底：回测显示紧急购电量从 39,212.5 kWh 降到 14,065.2 kWh，说明收缩情景覆盖并未转化为尾部风险的上升。

紧急购电的成因定位。全年 70 天出现紧急购电，合计 55,844.1 kWh，占计划购电量的 0.256%，单日紧急购电量的中位数为 0，最集中的两天是 2025-06-01 的 9,646.9 kWh 与 2025-06-02 的 8,003.5 kWh。成因是 5 月末连续两个低负载日之后，6 月负载出现约 50% 的季节跃升，超出了仅含 5 月的候选池所覆盖的范围。这说明紧急购电的触发条件是候选池的季节覆盖而非计划分位数本身，把池的月份环形距离设为 1 是在季节跃升边界上付出的代价。

7.4.3 结果与讨论

全年结果。2025-02-01 至 12-31 共 334 天，计划购电总量 21,850,664.3 kWh、计划购电费 16,814,531.4 元；紧急购电总量 55,844.1 kWh、紧急购电费 183,974.3 元；官方结算费用合计 16,998,505.7 元。334 天中有 70 天出现紧急购电，紧急购电量占计划购电量的 0.256%。储能日末储电量恒为下限 1200 kWh，日初储电量因此也恒为 1200 kWh，全年日内储电量在 1200 至 6609.6171 kWh 之间循环，从未触及上限。全年弃置电量 3,848,719.1 kWh，占计划购电量的 17.61%。

指定日期结果。下表按表 1 与表 2 的格式给出表 3 指定日期 2025.3.20、2025.6.21、

2025.9.23 与 2025.12.21 的结果，随后按表 3 的格式给出紧急购电结果。全部数值与结果文件逐项一致。表 1 的时段标签照抄附件 5 模板，数据按位置对齐，即模板第 k 个数据列对应附件第 k 个观测；表 2 填报的是实际执行路径，不带情景上标。

表 4 微网在指定时间段的购电量及全天的购电量和购电费

时间段	2025.3.20	2025.6.21	2025.9.23	2025.12.21
10:00-10:10	52.3188	3.1956	1.4019	383.4853
12:00-12:10	0.6003	0.0169	0.0000	146.4846
14:00-14:10	15.7876	6.8903	1.2945	289.4603
16:00-16:10	463.1452	118.6965	423.6913	760.4718
18:00-18:10	709.3962	431.3985	761.1071	723.0931
20:00-20:10	719.9573	627.2136	812.7410	723.1969
全天购电量	70659.3527	44844.3573	67970.5973	90849.7502
全天购电费	54644.6277	33288.6709	53225.9899	71365.2630

表中购电量单位为 kWh，购电费单位为元。全天购电费按计划购电量计算，即 $\sum_t P_t G_{d,t}$ 。四日的紧急购电量均为 0，故四日的官方结算费用与表 4 的全天购电费相同。

表 5 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	2025.3.20 充电/放电	2025.6.21 充电/放电	2025.9.23 充电/放电	2025.12.21 充电/放电
0:00-4:00	46.9488 / 38.0285	170.7311 / 129.2618	1130.8873 / 20.4117	747.9330 / 52.6090
4:00-8:00	62.6336 / 50.7332	67.1264 / 63.4028	180.2679 / 735.9268	1096.1437 / 157.6024
8:00-12:00	30.6536 / 24.8294	0.0000 / 0.0000	6.0645 / 310.6095	107.4785 / 1310.7401
12:00-16:00	344.5601 / 116.4673	0.0000 / 0.0000	2170.8368 / 859.2091	958.0034 / 630.3785
16:00-20:00	192.5584 / 219.7624	97.0170 / 78.5838	111.8732 / 852.9341	94.3075 / 261.0590
20:00-24:00	117.2726 / 193.8272	143.1059 / 115.9158	133.4679 / 244.9610	99.7832 / 101.5667
0:00 储电量	1200.0000	1200.0000	1200.0000	1200.0000
24:00 储电量	1200.0000	1200.0000	1200.0000	1200.0000

表中每个单元格的格式为充电量 / 放电量，电量单位为 kWh。四日的日内储电量区间分别为 1200.0 至 1510.1、1200.0 至 1287.3、1200.0 至 3148.6、1200.0 至 2766.7 kWh，全程落在 [1200, 10800] 内。

表 6 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0	无	0	无	0	无	0

四日在优化计划下均未触发紧急购电。四日的实际净负荷分别为 62,209.0、19,334.7、

62,116.6 与 89,037.8 kWh，均不超过当日计划购电量 70,659.4、44,844.4、67,970.6 与 90,849.8 kWh，因此 $E_{d,t} = 0$ 对所有时段成立，表 6 中相应填无。

跨日近视假设的检验结论。支持该假设的口头论证（见附录）在当前模型族内不可检验，本工作把当年跳过的检验补做完整，并在同一情景池、同一核权重、同一执行层下做配对对照：远视模型的全年官方结算费用为 17,036,556.63 元，比近视基线高 38,051 元，相对 -0.2233%，未达事先固定的 0.5% 采纳阈值，因此保留跨日近视假设与近视模型，负结果如实记录。这个负结果只在决策层远视而执行层近视这一配对口径下成立：它既不构成对跨日近视假设的证明，因为执行层的近视性是配对口径的一部分而非本次对照的发现，也不回答跨日近视究竟有多大代价。配对数据、负结果的传导机制与适用边界见附录 C。

八、 问题三模型的建立与求解

问题三在问题二的基础上引入光伏预报的日内更新：每天 0:00、6:00、12:00、18:00 均可获得未来 24 小时整点的光伏发电功率预报，微网可在 0:00 依据当时预报制定当天计划购电策略，也可在后续预报时刻调整尚未执行的购电量；计划购电量高于调整后购电量的部分按交易电价 50% 支付违约金，调整后高于计划的部分按 1.5 倍购买。从调度与决策的角度看，问题三的本质是一个**连续决策问题**：决策者需要在“修改计划要付调整费”与“不改计划可能要买 5 倍紧急电”之间反复权衡，储能设备则每 10 分钟依据实际负荷、实际光伏与当前储能能量闭环运行。因此本文采用**四阶段滚动随机规划 + 情景模型预测控制（MPC）**的整体框架：每个预报发布时刻求解一个条件两阶段随机规划，MPC 随新信息到来反复重算，四次顺序求解共同构成一条可实施的闭环策略。

8.1 数据预处理与探索性分析

对四份附件进行核对发现，各数据均无缺失值、无异常值、无量纲不一致或单位错乱，数据结构完整且相互对应，因此无需专门预处理，可直接用于建模；唯一需要注意的是附件给出的 144 个十分钟区间标签与结果模板存在十分钟错位，本文按位置对齐处理。

建模之前，我们先对负荷、光伏及二者关系作探索性分析。观察负荷数据可以发现，小区负荷呈现清晰的**日内双峰形态**（早晚两个高峰、中午凌晨较低），且**周同期**（上周同一时刻）是最强的预测线索，其加权绝对百分比误差明显低于前一日同期；这种“日内有规律、周内可借鉴”的周期性提示负荷预测应以周同期为基线。光伏出力主要由日照时长与太阳高度角决定，呈现强烈的**季节性**：夏季高峰更高、持续更长，冬季显著偏低，全年净负荷日均值在冬季约为春季两倍。将负荷与光伏相减得到净负荷后进一步发现，二者在**时间上并不匹配**——全年有近两成的十分钟时段光伏功率超过负荷、盈余集中在正午，而负荷高峰在早晚，这一错配恰恰揭示了储能“时间上的能量搬运”价值。综上，负荷的周同期周期性、光伏的季节性、以及负荷与光伏的时序错配，共同构成建模的三个关键前提。

8.2 模型建立

8.2.1 符号设定与基本假设

本文的基本假设为：一天 144 个十分钟区间、能量以 kWh 计量；问题三电价 $P_t > 0$ 为附件 1 固定日内曲线；用户侧不能反向售电、光伏过剩可弃光；更新只对未开始区间生效；主口径按“最终调整量相对 0:00 原计划结算一次”；微网在每个区间内可连续计量并闭环调节。

8.2.2 负载与光伏的因果预测

基于探索性分析发现的周周期特性，负载预测采用“周同期基线 + 同阶段缩放”：基线取上周同一时刻负载功率 $\ell_{d-7,t}$ （不足 7 天用附件 1 平均曲线），缩放系数 $a_{d,k}$ 用最近至多 36 个已观测区间的中位比例估计并以历史 5%/95% 分位截断，预测为 $\hat{L}_{d,k,t} = \Delta \max\{0, a_{d,k} m_{d,t}\}$ 。光伏预测则把附件 3 的整点预报加上发布时刻实测功率作为节点，用保形分段三次插值（PCHIP）构造连续曲线并逐十分钟积分得到 $\hat{R}_{d,k,t}$ 。所有预测严格只用当前发布时刻之前的信息，杜绝数据泄漏。

8.2.3 联合不确定性情景

考虑到负载与光伏的预测误差在日内高度相关，情景生成必须保留同期相关性。对历史日 $j < d$ 因果重放预测得到残差 $\varepsilon_{j,t}^L = L_{j,t} - \hat{L}_{j,t}$ 、 $\varepsilon_{j,t}^R = R_{j,t} - \hat{R}_{j,t}$ ，再从与 d 同季节的历史日中有放回抽取 N_ω 个、取同一历史日的整段残差构造情景

$$L^\omega = \max\{0, \hat{L} + \varepsilon^L\}, \quad R^\omega = \max\{0, \hat{R} + \varepsilon^R\}, \quad (25)$$

等权概率 $\pi = 1/N_\omega$ 。同季节条件化既反映季节差异，又避免跨季节抽样引入错误幅度的误差。

8.2.4 供需平衡与储能模型

对任意情景 ω 、区间 t ，能量平衡与储能量递推为

$$G_{d,t}^{(k)} + E_{d,t}^\omega + R_{d,k,t}^\omega + D_{d,t}^\omega = L_{d,k,t}^\omega + C_{d,t}^\omega + W_{d,t}^\omega, \quad (26)$$

$$S_{d,t+1}^\omega = S_{d,t}^\omega + \eta_c C_{d,t}^\omega - \frac{D_{d,t}^\omega}{\eta_d}. \quad (27)$$

式 (26) 左边为供给侧（常规购电、紧急购电、光伏、储能放电），右边为需求侧（负载、储能充电、弃置）。由于紧急购电单价为 $5P_t$ 且常规购电无上限，最优解中紧急电只用于真实负载缺口，故单一弃置量 W 即可吸收所有未利用电量而不改变最优费用。容量与功率边界为 $1200 \leq S_{d,t}^\omega \leq 10800$ 、 $0 \leq C_{d,t}^\omega, D_{d,t}^\omega \leq \bar{e} \approx 833.33$ 。

8.2.5 计划购电与调整费用

计划购电是非预期的第一阶段变量，对所有情景取同一值。引入绝对调整量 $A_{d,t}^{(k)} \geq 0$ 满足 $A \geq G^{(k)} - G^{(0)}$ 与 $A \geq G^{(0)} - G^{(k)}$ ，最优时自动压到 $A = |G^{(k)} - G^{(0)}|$ ，从而把绝

对值线性化。单区间常规购电与调整的合计费用为

$$\Phi_{d,t}^{(k)} = P_t G_{d,t}^{(k)} + 0.5 P_t A_{d,t}^{(k)}, \quad (28)$$

经恒等变换，这与题面“上调部分按 1.5 倍、下调违约 0.5 倍”完全等价。

8.2.6 报童下界与跨日终端价值

规划层的第二阶段补救按情景自由取值，会高估储能的对冲能力、把计划购电量压得偏低。为对冲这一乐观偏差，对每个区间增加报童下界：计划购电量不得低于情景净负载的 80% 分位，

$$G_{d,t}^{(k)} \geq Q_{0.8}(\{L_{d,k,t}^\omega - R_{d,k,t}^\omega\}_\omega), \quad (29)$$

其理论依据是单区间报童模型的临界比：少买 1 kWh 多付 $4P_t$ 、多买 1 kWh 白付 P_t ，故最优覆盖概率为 0.2。

若只最小化当天费用，模型会在 24:00 前放空储能、次日丧失隔夜缓冲。为维持隔夜缓冲，构造凸分段线性终端价值 $V(s)$ ，在安全区间取 17 个网格点、对每个网格点用次日点预测解确定性 24 小时影子调度得未来费用，再投影到凸集合重建 $V(s) = \max\{0, \max_q(\tilde{a}_q s + \tilde{b}_q)\}$ ，并线性化为 $\theta^\omega \geq \tilde{a}_q S_{d,145}^\omega + \tilde{b}_q$ 。 θ 只用于防止短视，不计入官方结算。

8.2.7 目标函数与执行层情景 MPC

第 k 次更新时，情景 ω 的待决策成本与完整目标为

$$J_{d,k}^{M,\omega} = \sum_{t \in \mathcal{R}_k} \left[P_t G_{d,t}^{(k)} + 0.5 P_t A_{d,t}^{(k)} + 5 P_t E_{d,t}^\omega \right] + \theta_{d,k}^\omega, \quad (30)$$

$$\min (1 - \lambda) \sum_{\omega} \pi_{\omega} J_{d,k}^{M,\omega} + \lambda \text{CVaR}_{\alpha}(J_{d,k}^M). \quad (31)$$

默认取风险中性 $\lambda = 0$ 。由于规划层的 **wait-and-see** 是乐观下界，因果可实施性由执行层保证：实际运行到区间 r 时，冻结已提交的 G^F ，重采样未来情景，解“当前区间 **here-and-now**、未来 **wait-and-see**”的随机 LP

$$\min_{\omega} \sum_{\omega} \pi_{\omega} \left[\sum_{t=r}^{144} 5 P_t E_{d,t}^\omega + \theta^\omega \right], \quad (32)$$

只执行当前动作，并对当前区间施加互斥约束（保证不充放、紧急电不充电），下一区间重新采样重解，保持滚动因果性。

8.3 模型求解

规划层与执行层的目标函数均为线性函数，全部约束为线性等式或不等式，报童下界与终端价值的线性化不改变模型的线性结构，故每个子问题都是连续线性规划，可直接由线性规划求解器精确求解；本文统一采用 SciPy 的 `scipy.optimize.linprog` (HiGHS 方法)。求解按天滚动、每天四阶段顺序执行：

1. **Step 1 (0:00 制定原计划)**: 读取当前储能量与未来 24 小时光伏预报, 建立因果负载/光伏预测与联合情景, 求解两阶段随机 LP, 得到全天原计划 $G^{(0)}$;
2. **Step 2 (区内闭环执行)**: 进入某一 10 分钟区间前, 冻结已提交的 G^F , 重采样未来情景并求解情景 MPC, 只执行当前区间的充放电与紧急购电动作, 下一区间重新采样重解, 保持滚动因果性;
3. **Step 3 (6:00、12:00、18:00 更新)**: 冻结已执行时段, 读取新预报与实测储能量, 重建情景并重解剩余时域, 得到更新后的总购电量 $G^{(k)}$;
4. **Step 4 (日末衔接)**: 24:00 保存真实执行路径, 令次日初值等于当日末储能量, 返回 Step 1 推进下一天。

四种更新策略 $M_0 = \{0\}$, $M_1 = \{0, 6\}$, $M_2 = \{0, 6, 12\}$, $M_3 = \{0, 6, 12, 18\}$ 从同一公共初始储能量分别滚动全年, 进程级并行以减小墙钟时间。

8.4 结果分析

8.4.1 模型验证

模型的正确性通过三类残差复核: 能量平衡残差、储能量递推残差、费用复算残差, 实测分别约 4.6×10^{-13} 、 9×10^{-13} 、 3.7×10^{-9} , 均远小于验收阈值, 且无越界、无同时充放电、无紧急购电充电, 说明模型求解稳定、物理口径自洽。

为评价各更新时刻的价值, 本文将四种更新策略在同一公共初始储能量下分别滚动全年, 比较其官方结算费用, 结果如表 7 所示。

表 7 四种更新策略的全年官方费用 (万元)

策略	总费用	调整费贡献	ΔC
M_0 (仅 0:00)	1795.6	—	—
M_1 (+6:00)	1908.1	110.3	$\Delta C_6 = -112.5$
M_2 (+12:00)	1882.4	127.4	$\Delta C_{12} = +25.8$
M_3 (+18:00)	1877.6	126.2	$\Delta C_{18} = +4.7$

表 7 显示, 仅保留 0:00 计划的 M_0 总费用反而最低, 依次加入 6:00、12:00、18:00 更新后总费用不降反升。为看清总费用由何驱动, 图 9 给出各策略的计划购电费、调整费与紧急购电费的分解。

观察图 9 可以发现, 四种策略的紧急购电费均处于较低水平 (得益于报童下界与情景 MPC 的双重对冲), 总费用主要由计划购电费主导; 而 M_1, M_2, M_3 相对 M_0 多出的调整费, 正是其总费用偏高的直接原因。这一结果提示: 日内更新虽能改善计划精度, 却要额外支付调整费, 其经济性取决于“精度提升”与“调整代价”的权衡。

8.4.2 关键设计取舍的敏感性分析

本文的三个关键设计取舍对结果影响显著, 采用控制变量对照 (每次只改变一个因素) 逐一探讨, 统一以 M_3 策略的全年总费用与紧急购电量作为评价指标。

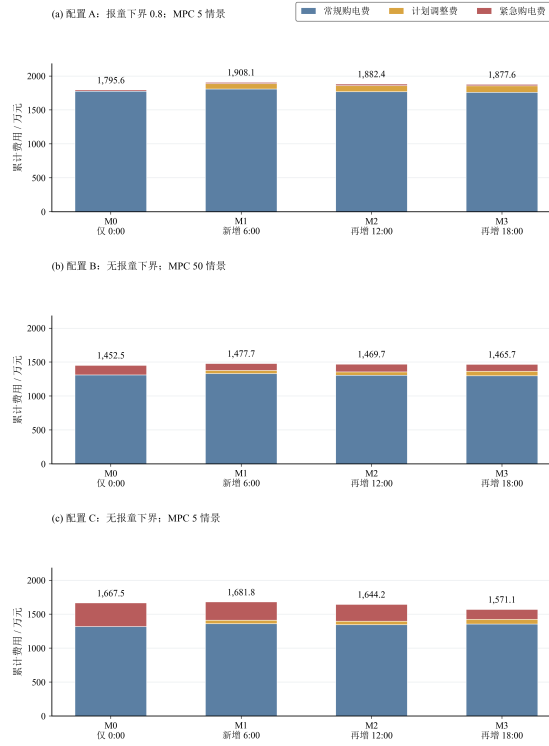


图 9 四种策略的费用构成

取舍一：是否保留终端价值 θ 。在相同情景、相同种子下仅切换是否加入 θ ，结果表明删 θ 后储能每晚放空到安全下限、紧急电暴涨约 4.6 倍、总费用上升约 9%。这说明储能的隔夜缓冲价值远大于单纯的电价套利价值， θ 必须保留。

取舍二：是否保留报童下界。控制变量对照（下界 0.8 vs 无下界，MPC 均为 5）表明，报童下界把紧急电压低 78%，但以计划购电严重超购为代价，总费用反而上升 19.5%。这反映了紧急电与总费用之间此消彼长的 trade-off， $q=0.8$ 是“最保守”而非“最优”。

取舍三：MPC 情景数。控制变量对照（MPC 5 条 vs 50 条，均无下界）表明，增加执行层情景数使储能决策更充分地对冲未来不确定性，总费用与紧急电同步下降、无副作用，属于净收益，其代价仅是计算时间约增 14 倍。

8.4.3 小结

问题三本质是“计划购电 + 日内调整 + 储能闭环”的连续决策问题，本文用四阶段滚动随机规划 + 情景 MPC 统一建模。通过控制变量对照得到三个明确结论：跨日终端价值 θ 不可省略；报童下界 $q=0.8$ 过保守；执行层情景数 5→50 为净收益。关于“6:00 更新有害”这一现象，本文的理解是：它并非运行噪声，而是在多种配置下稳定出现，其深层原因在于 0:00 预报已覆盖全天、执行层情景 MPC 每 10 分钟依据实测状态闭环响应，使“计划购电量的精确性”对最终费用的影响被削弱，此时 6:00 再调整只能节省有限购电费、却要支付 0.5 倍调整费，边际上得不偿失；相对地，12:00 与 18:00 更新能有效利用当日后半段的负荷与光伏信息，仍具正价值。这一现象提示：在引入了日内闭环储能响应的框架下，更新时刻的价值取决于“新增信息量”与“调整代价”的权衡。

九、 问题四模型的建立与求解

问题四把问题三的固定电价改为**实时波动的电价**（附件 4），要求在波动电价下重新计算问题二与问题三：问题 4-2 每天 0:00 只制定一次计划购电量、日内不改；问题 4-3 每天 0:00 制定原计划，6:00、12:00、18:00 根据新光伏预报与最新价格信息调整尚未执行的购电量。问题四相对问题三的实质新增在于**电价的不确定性**：查阅相关资料可知，电价本质上反映电力供需关系，因而同时受日内负荷波动、工作日与周末用电结构差异、季节性负荷等多重时间尺度影响，故不能把固定价数组简单替换成均值，而必须显式刻画电价的多尺度规律并将其不确定性纳入随机规划。除电价外，其余结构（单 W 能量平衡、SOC 递推、调整结算、跨日终端价值 θ 、情景 MPC、报童下界、同季节条件化）与问题三完全一致，本文不再重复。

9.1 数据预处理与探索性分析

与问题三相同，附件 4 的波动电价数据经核对**无缺失值、无异常值、无量纲不一致**，数据完整可用，无需专门预处理。其电价范围为 0.0076–1.7936 元/kWh，全年每日峰谷价差中位数约 1.056 元/kWh，说明价格波动幅度可观、峰谷价差足以显著改变储能充放电的先后顺序。

考虑到电价受多尺度因素共同驱动，我们对电价作三层分解。观察电价的日内曲线可以发现，电价呈现与负荷类似的**峰谷形态**：白天用电高峰对应电价高位、凌晨低谷对应低位，这一“昼夜”规律源于电力供需的日内周期波动，也意味着储能存在明显的日内套利空间。进一步观察发现，电价还表现出明显的**周周期**：上周同一时刻的价格是本周价格的最强预测线索（滚动 WAPE 约 6.22%），且工作日与周末价格水平存在系统性差异，符合工商业负荷集中于工作日的常识。电价水平还随季节缓慢漂移，冬季与夏季因取暖、制冷负荷而整体偏高。综合以上三个尺度，电价可分解为“季节基线 + 周内偏移 + 日内峰谷”的叠加结构，这正是电价预测模型的设计依据。

9.2 模型建立

9.2.1 符号设定

沿用问题三全部符号，新增电价相关符号： $P_{d,t}$ 为实际波动电价， $\hat{P}_{d,t}$ 为电价预测， $P_{d,t}^\omega$ 为情景电价， $m_{d,t}^p$ 为周同期基线， $\bar{P}_{d,t}^{\text{seas}}$ 为季节基线， $\gamma_{d,t}^p$ 为日内时段效应， $\delta_{d,r}^p$ 为水平修正， $\varepsilon_{j,t}^p$ 为电价预测残差。

9.2.2 电价三尺度因果预测

基于探索性分析的三尺度结构，电价预测依次叠加四部分。季节基线取同季节历史日均价曲线 $\bar{P}_{d,t}^{\text{seas}}$ ；周同期基线取上周同一时刻价格 $m_{d,t}^p = P_{d-\tau,t}$ （冷启动用季节基线）；日内时段效应 $\gamma_{d,t}^p$ 从同星期历史日的周残差中估计并经零和化，只描述昼夜中稳定的峰谷偏移；水平修正 $\delta_{d,r}^p$ 用最近至多 36 个已实现价格估计当天整体水平并截断。最终电价预测为

$$\hat{P}_{d,t} = \max \left\{ \varepsilon_p, m_{d,t}^p + \gamma_{d,t}^p + \delta_{d,r}^p \right\}. \quad (33)$$

三尺度对应关系清晰：昼夜由 m^p 的日内形状与 γ^p 的日内峰谷共同描述，周内由 $m^p = P_{d-7,t}$ 的周周期描述，季节由 $\bar{P}^{\text{seas}}$ 与 δ^p 描述。

9.2.3 负载—光伏—电价联合情景

对历史日 $j < d$ 因果重放三类残差 $\varepsilon^L, \varepsilon^R, \varepsilon^p$ ，再从与 d 同季节的历史日中抽取同一历史日的整段残差构造三元情景

$$L^\omega = \max\{0, \hat{L} + \varepsilon^L\}, \quad R^\omega = \max\{0, \hat{R} + \varepsilon^R\}, \quad P^\omega = \max\{\varepsilon_p, \hat{P} + \varepsilon^p\}. \quad (34)$$

三类残差必须来自同一历史日整段块，以保留“高负荷、低光伏、高电价”之间的同期相关性。

9.2.4 优化模型（价格情景化）

问题四的物理约束与问题三完全相同，仅把目标与结算中的固定价换成情景价（优化）或实际价（结算）。问题 4-3 第 k 次更新的目标为

$$\min \sum_{\omega} \pi_{\omega} \left[\sum_{t \in \mathcal{R}_k} (P_{d,t}^{\omega} G_{d,t}^{(k)} + 0.5 P_{d,t}^{\omega} A_{d,t}^{(k)} + 5 P_{d,t}^{\omega} E_{d,t}^{\omega}) + \theta^{\omega} \right]. \quad (35)$$

执行层情景 MPC 的目标相应为 $\min \sum_{\omega} \pi_{\omega} [\sum_{t=r}^{144} 5 P_{d,t}^{\omega} E_{d,t}^{\omega} + \theta^{\omega}]$ ，当前区间电价用实际价、未来电价用情景价。官方结算用真实价格 $P_{d,t}$ 复算。

9.3 模型求解

问题四沿用问题三的滚动求解流程、Step 式步骤与并行结构，仅在预测层增加电价三尺度预测、在情景层增加电价一维；各子问题仍为连续线性规划，统一采用 SciPy 的 `scipy.optimize.linprog` (HiGHS 方法) 求解。电价情景只是在情景矩阵中多一维 P^{ω} ，不改变 LP 规模，故每策略计算成本与问题三同级。四组配置 (C2、C3、PI2、PI3) 进程级并行。

9.4 结果分析

9.4.1 模型验证

能量平衡、SOC 递推、费用复算三类残差均通过（分别约 4.5×10^{-13} 、 9×10^{-13} 、 3.7×10^{-9} ，无违规），说明波动电价下的求解同样稳定。

问题四的官方结算费用与价格信息价值见表 8。价格信息价值定义为有限年度费用差 $\widehat{\Delta C}^p = C_{\text{因果}} - C_{\text{完美价格}}$ ，即“提前知道当天完整电价”相对“因果预测电价”能节省的费用。

观察表 8 可以发现：在报童下界 $q=0.8$ 时价格信息价值几乎为零（甚至略负），关掉下界后转为正值，且随 MPC 情景数增加而进一步增大。这一规律具有明确的经济含义——计划越保守，越不需要“精确价格”这一额外信息；只有当计划真正依赖价格预测时，完美价格信息才能转化为实际经济收益。可见价格信息的价值并非固定，而是由决策框架的对冲程度内生决定。

表 8 问题四官方结算费用与价格信息价值（万元）

配置	C2（4-2 因果）	C3（4-3 因果）	信息价值 4-2	信息价值 4-3
下界 0.8、MPC5	1862.8	1939.9	-0.7	-0.03
无下界、MPC5	1710.5	1628.8	+4.4	+14.1
无下界、MPC50	1518.5	1533.6	+14.2	+16.2

除价格信息价值外，波动电价对微网的运行形态也产生了可观测的影响。图 10 给出固定电价（问题三）与波动电价（问题四 4-2、4-3）下月度购电费用与紧急购电需求的对比。

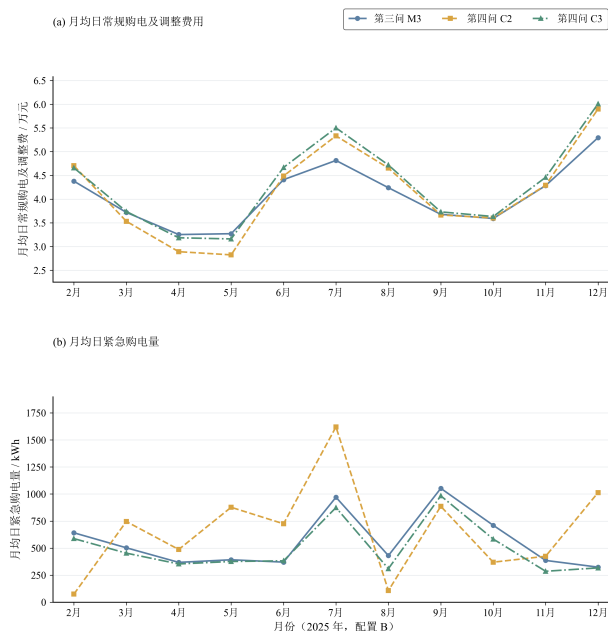


图 10 固定电价与波动电价下月度购电费用与紧急购电需求

观察图 10 可以发现，两类电价下的购电费用与紧急购电需求都保持相似的季节性轮廓——冬季负荷高、光伏低时抬升，但波动电价使各月的购电费用出现重新排序：价格高企的月份购电费用相对上升，价格低谷的月份则相对下降，说明电价的多尺度波动通过改变储能套利与计划购电的时机，重塑了全年费用的月度分布。图 11 进一步从日内与日末两个维度刻画储能运行的变化。

观察图 11 可以发现，储能依然保持“充电与放电交替”的日内套利形态，但波动电价下充放电的时段位置更贴近电价峰谷而非单纯负荷峰谷；同时日末储能量仍稳定维持在安全区间中高位，说明终端价值 θ 在波动电价下同样维持了隔夜缓冲。可见，价格信号与负荷/光伏信号共同作用于储能，储能既削峰填谷又套利，但并未因价格波动而破坏跨日保电的稳健性。

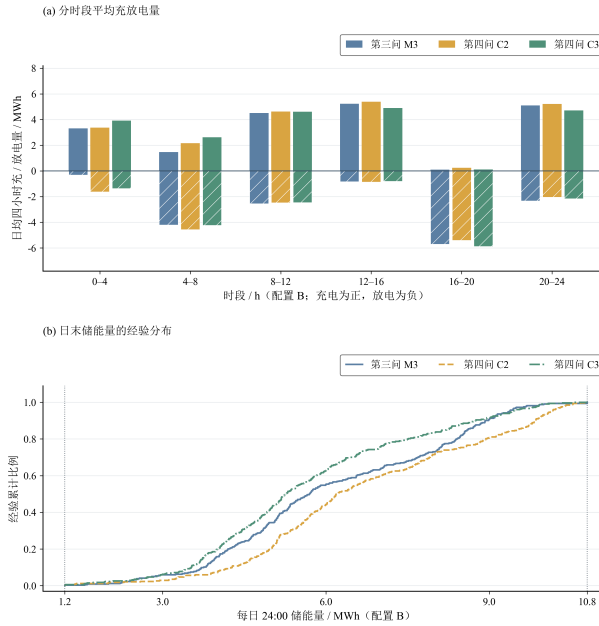


图 11 固定电价与波动电价下储能日内充放电与隔夜储能

9.4.2 价格信息口径的敏感性分析

本文区分“因果价格预测”与“完美价格信息 (PI-DA, 每天 0:00 已知当天全价)”两种口径：前者是实际可执行的方案，后者是信息基准。如模型验证所示，两者费用差随计划保守程度递减，说明在本文的滚动随机规划框架下价格信息的边际价值有限；但这一结论依赖报童下界与情景 MPC 的对冲强度，不能外推到更简单的确定性模型。

9.4.3 小结

问题四把问题三的固定价换成波动价：先用“季节基线 + 周同期基线 + 日内时段效应 + 水平修正”显式刻画电价的昼夜、周内、季节三尺度规律，再与负载、光伏一起抽成同一历史日的三元联合残差情景，喂给与问题三完全相同的两阶段随机规划 + 情景 MPC + 报童下界 + 终端价值框架，最终用真实价格结算。价格信息价值实验表明其经济价值随计划保守程度递减，这既印证了模型对冲机制的合理性，也为“是否需要更精确的价格预测”提供了可量化的判断依据。

十、 模型的评价与推广

10.1 模型优点

1. **框架统一、可递进。**以“情景生成 + 两阶段随机规划 + 执行层闭环”为主线，四个问题只改变不确定性来源，物理约束始终一致，便于横向比较。
2. **决策口径严谨。**情景只取决策日之前的历史日，第一阶段变量全情景共享，共同出发点约束封堵起点套利，扰动检验表明无前视泄漏。
3. **对冲与校正并重。**报童下界抑制补救层的乐观偏差，终端价值 θ 维持隔夜缓冲，充

电来源约束消除紧急电套利。

4. 结果可信、对照量化。各项残差仅 10^{-9} 量级，无越限与同时充放电；相对无储能方案，购电费用降低 26.92%。
5. 含反直觉发现。更新预报并非越频繁越好（6:00 边际收益为负），价格信息的价值随计划保守程度递减，均有对照实验支撑。

10.2 模型缺点

1. 情景池受历史覆盖限制。候选池的月份范围决定精度，负载季节跃升时会触发紧急购电；核加权使有效情景数偏少，可能低估尾部风险。
2. 保守设定代价明显。第二问模型全年弃置电量占计划购电量的 17.61%，报童下界取 0.8 分位使总费用上升约 19.5%，偏保守而非最优。

10.3 模型的改进方向

针对上述缺点，可从以下两方面改进：

1. 提高情景池的覆盖能力。引入季节跃升检测，在负载或净负荷突变时动态扩大候选池的月份环形距离或叠加趋势项，避免季节跃升边界出现紧急购电；用 CVaR 或更重尾的情景构造补充核加权对尾部风险的低估。
2. 以费用口径自适应选定保守参数。报童下界分位数 q 与核带宽 h 目前分别独立选取，可在统一的费用口径下交叉验证、联合搜索，用实际结算费用作为选参标准，避免固定取 0.8 分位带来的过度保守。

10.4 模型的推广

本文提出的“情景生成 + 两阶段随机规划 + 情景 MPC + 终端价值 + 报童下界”框架，本质上是含储能设备、面临多重时序不确定性、需要日内滚动决策的联合调度问题的一般解法。这一思路可直接迁移到含储能的虚拟电厂调度、风光储综合能源系统运行、用户参与电力现货市场与需求响应等场景：只需把“负载—光伏”的不确定性替换为风电或价格等随机量，把“购电 + 调整 + 紧急购电”的结算规则替换为对应市场的出清与偏差考核规则，即可复用本模型。这使本文工作不仅回答题目四问，也为电力市场环境下的储能经济调度提供了可落地的建模范式。

参考文献

- [1] U.S. DEPARTMENT OF ENERGY. Solar Integration: Solar Energy and Storage Basics[EB/OL]. (2025-03-31)[2026-09-13].
- [2] U.S. ENERGY INFORMATION ADMINISTRATION. Homes show greatest seasonal variation in electricity use[EB/OL]. (2013-03-04)[2026-09-13].
- [3] NGUYEN T N, MÜSGENS F. What drives the accuracy of PV output forecasts?[J]. Applied Energy, 2022, 323: 119603.

- [4] U.S. ENERGY INFORMATION ADMINISTRATION. Factors affecting electricity prices[EB/OL]. [2026-09-13].
- [5] 肖浩, 裴玮, 孔力. 基于模型预测控制的微电网多时间尺度协调优化调度 [J]. 电力系统自动化, 2016, 40(18): 7–14, 55.
- [6] 袁泉, 吴云亮, 李豹, 等. 计及源荷不确定性的多时间尺度滚动调度计划模型与方法 [J]. 电力系统保护与控制, 2019, 47(16): 8–16.
- [7] 蔡瑶, 卢志刚, 潘尧, 等. 计及多重差异的交直流混合多能微网多时间尺度优化调度 [J]. 电工技术学报, 2024, 39(11): 3392–3410.

支撑材料的项目架构

支撑材料包含：按题目要求上传的各 result 文件，按小问划分的三个相互独立的求解工程，另附一份 AI 工具使用情况说明。Q1 求解处理典型日的购电与储能联合调度，输入附件 1，产出调度明细、结果表与四组图件，并以 Markdown 给出该问的章节初稿。Q2 求解采用同一套“输入数据—求解代码—数值产物—图件”的四层结构：根目录放统一符号设定与研究日志，solving/ 下按核赋权原型、近视基线定稿、远视化配对对照三个阶段各自成包，outputs/ 与 assets/ 分别存放三个阶段的结果与图件，doc/、task/ 与 utils/ 收录数据探索记录、任务书与辅助代码。Q3,Q4 求解以模块化包结构实现滚动决策与波动电价下的重算，代码、建模文档与结果表分别放在 src/、doc/ 与 output/。三个工程都只依赖题目附件，可分别独立运行与复现，论文正文中的每一项数值都能在对应工程的输出目录中找到落盘文件。

跨日近视假设的论证史与实验证据

本附录是问题二正文的配套材料，承载正文因篇幅与叙事纪律而压缩的全部论证过程、模型构造细节与实验数据。正文与附录的引用关系见表 9，每一处正文引用都能在本附录中找到对应章节。

表 9 正文引用与附录章节的对应关系

正文位置	正文中的表述	附录对应章节
问题分析·一个假设与一个挑战	口头论证的命题链在当前模型族内不可检验，详见附录 A	附录 A 与附录 B
模型建立·跨日近视假设的检验设计	完整的论证史、模型构造细节与全部实验数据见附录 A 至附录 C	附录 A、附录 B、附录 C
数据预处理·特征筛选	事后候选的标注与证据强度说明见附录 D	附录 D
敏感性分析·核加权方案与带宽	带宽搜索过程与三套方案对比	附录 D
模型验证·一处需要如实记录的模型性质	弃置电量的成因与量级	附录 A 第三节与附录 C 第五节
结论·跨日近视假设的检验结论	配对数据、负结果的传导机制与适用边界	附录 C

本附录中标注为远视侧的全部数值均取自 outputs/taskCompositeFar/，标注为近视侧的全部数值均取自 outputs/taskComposite/，不含任何未落盘的中间量。每条结论后面均标注对应的输出文件路径，可逐项复算。

附录 A 跨日近视假设的论证史

A.1 原口头论证与其命题链

跨日近视假设在建模初期的支持依据是一段口头论证，其核心判断是日末储能的边际价值被封顶在次日凌晨的低谷电价附近。该论证给出的理由有三条：电价低谷恰好横跨日界，22:00 至次日 5:00 的购电价格在 0.42 元/kWh 附近；凌晨可以重新购入电量，储能充满 9600 kWh 可用容量只需约 12 个时段，替代供给在功率侧充足；凌晨净负荷日均 17,476.5 kWh，远超 9600 kWh 的可用容量，替代供给在电量侧同样充足。由此推出

两端电价几乎相等，最优解处于放掉与留下近乎无差异的临界，两个模型的日末行为趋于一致，跨日近视假设可以接受。

把这段论证重写为可逐条判定的命题链，可得到七条命题。命题的检验状态如表 10，逐条复算的完整数值见 `outputs/taskCompositeFar/assumption_audit.md` 与 `outputs/taskCompositeFar/assumption_audit.json`。

表 10 跨日近视口头论证的命题链与检验状态

编号	命题	检验状态
P1	近视模型在 22:00—24:00 的低谷窗口把储能放空	既有数据支持
P2	假设成立的充分条件为 $\max_q(-a_{d,q}) \lesssim \min_{t \in \mathcal{T}^{\text{晚}}} P_t$	不可检验
P3	日末储能的边际价值被封顶在凌晨低谷价约 0.42 元/kWh 附近	不可检验
P4	附件 1 在 22:00—24:00 的电价为 0.418—0.427 元/kWh	既有数据与预期不符
P5	两端几乎相等推出两模型日末行为趋于一致，进而接受跨日近视假设	不可检验
P6	计划购电直达负载且无功率上限，凌晨空仓只影响套利机会而不阻塞供电	既有数据与预期不符
P7	凌晨紧急购电是全天最便宜的紧急购电时段，可用计划裕度廉价对冲	既有数据支持

A.2 命题链的逐条复算

P1 检验状态为既有数据支持。日末储电量恒为下限的天数为 334/334，总体标准差为 0.000000 kWh；晚高峰 22:00—24:00 的放电总量为 47,079.635282 kWh、充电总量为 20,182.174002 kWh。行为描述与观测一致，但该描述只刻画近视模型自身的最优解，不含任何关于远视模型会如何行动的信息。出处 `outputs/taskComposite/dispatch_year.csv` 的 `storage_end` 列与 `outputs/taskComposite/dispatch_detail.csv`。

P2 检验状态为不可检验。左端对象是终端价值函数 $V_{d+1}(s)$ 的分段线性下界的切线斜率 $a_{d,q}$ ，既有产物中记录该量的文件为无，右端即附件 1 晚高峰窗口电价为 0.385900 与 0.436800 元/kWh。 $a_{d,q}$ 是远视模型决策约束 $\theta_d^\omega \geq a_{d,q} S_{d,145}^\omega + b_{d,q}$ 的系数；在近视模型族中 θ 不进入目标函数，没有任何最优性条件涉及 $a_{d,q}$ 。形式化论证见附录 B。

P3 检验状态为不可检验。该命题包含事实层与断言层，两者地位不同。事实层可由附件 1 与既有产物完全复算，且复算结果与原论证一致：晚高峰电价区间 0.385900 至 0.436800 元/kWh、均值 0.421533；凌晨 0:00—5:00 电价区间 0.424500 至 0.442200 元/kWh、均值 0.433393；凌晨均价比晚高峰均价高 2.813538%；凌晨净负荷日均总量 17,476.499469 kWh；次日凌晨净负荷总量低于可用容量 9600 kWh 的天数为 0/333；可用容量 9,600.000000 kWh；充电功率上限 833.333333 kWh/10min；充满可用容量所需时段数 11.520000。断言层即边际价值被封顶在 0.42 附近这一陈述，是关于 $V_{d+1}(s)$ 斜率的陈述，在当前模型族内没有定义，不可检验。复算同时显示两端几乎相等这一判断掩盖了一个方向性不对称：凌晨电价均值反而高于晚高峰均值，且晚高峰窗口内的最低价 0.3859 低于凌晨最低价 0.4245。出处 `outputs/taskCompositeFar/assumption_audit.md` 与 `ProblemC/附件/附件 1.xlsx`。

P4 检验状态为既有数据与预期不符。原论证引用的区间为 0.418000 与 0.427000 元/kWh，实际复算的区间为 0.385900 与 0.436800 元/kWh，均值为 0.421533；全天最低电价 0.371300 元/kWh 出现在 5:40；低于 0.418 元/kWh 的晚高峰时段数为 1。原论证引用的下界 0.418 元/kWh 过低，这直接影响两端几乎相等这一判断的分辨率。出处 `ProblemC/附件/附件 1.xlsx`。

P5 检验状态为不可检验。该推理链的承重点是 P2，而 P2 不可检验，因此推理链在承重处断开。两端几乎相等只是对两个未经独立计算的数字的比较。被跳过的实验是 $V(s)$ 扫描实验。出处 `outputs/taskCompositeFar/assumption_audit.md`。

P6 检验状态为既有数据与预期不符。全年弃置电量 3,848,719.116083 kWh，占计划购电比例 17.613740%，为充电量的 6.165750 倍，出现弃置的天数 333/334；计划购电超出实际净负荷 3,911,474.791157 kWh；净负荷为正时段的弃置电量 855,035.905648 kWh；充电已达功率上限时段的弃置电量 1,172.468069 kWh，充电未达功率上限时段的弃置电量 3,847,546.648014 kWh，后者占弃置总量的 99.969536%；充电已达功率上限的时段数为 57；储能全天最高储电量在 365 天口径下为 6,609.617145 kWh，距储能上限 10800 kWh 的余量为 4,190.382855 kWh；充电日均利用率占可用容量 9600 kWh 为 0.194676。计划购电直达负载并不成立，因为全年有 17.613740% 的计划购电量被弃置；这些弃置既不能由充电功率上限解释，也不能由储能容量解释，只能由多余的来源在当天无处可用且在目标函数中不被跨日赋值解释，而后者正是跨日近视假设本身。出处 `outputs/taskComposite/dispatch_year.csv` 与 `outputs/taskComposite/dispatch_detail.csv`。

P7 检验状态为既有数据支持。凌晨 0:00—5:00 的紧急购电量为 16,218.255333 kWh，晚高峰 22:00—24:00 的紧急购电量为 5,006.120080 kWh，凌晨紧急购电占全年比例 29.042016%，全年紧急购电量 55,844.109499 kWh，出现紧急购电的天数 70/334。既有结果显示凌晨几乎不触发紧急购电，与该命题方向一致，但它同样只说明近视模型的行为，不构成对 P2 左端的支持。出处 `outputs/taskComposite/dispatch_detail.csv`。

A.3 与论证预期不符的定量信号

以下四组信号全部可由 `outputs/taskComposite/` 的既有文件直接复算。必须先声明的一组不成立的推论是：这些现象并不能证明跨日近视假设为假。它们与近视模型的最优性完全一致，是同一假设的推论而非反驳；用假设的推论去否定假设，与用假设本身支持假设在逻辑上同样无效。这些信号的作用是定位原论证的薄弱环节，并说明 $V(s)$ 扫描检验必须补做。

信号一：储能容量与充电功率都不是瓶颈，日末空仓不是约束的强加。日末与日初储电量在 334 天全部等于下限 1200 kWh，标准差为 0；储能全天最高在 365 天口径下为 6,609.6171 kWh，距上限 10800 kWh 尚有 4,190.3829 kWh，全年没有任何时段触及上限；全年充电量 624,209.4 kWh，日均 1,868.9 kWh，只占可用容量 9600 kWh 的 19.468%。这组事实说明日末空仓并非储能装不下，而是目标函数的选择。出处 `outputs/taskComposite/dispatch_year.csv` 的 `charge_kwh`、`storage_start`、`storage_end` 列与 `outputs/taskComposite/validation_daily.csv` 的 `storage_max_kwh` 列。

信号二：弃置电量既不能由充电功率上限解释，也不能由储能容量解释。全年弃置 3,848,719.1 kWh，占计划购电量的 17.61%，是全年充电量的 6.166 倍，333/334 天出现弃置；计划购电总量超出实际净负荷 3,911,474.8 kWh，占计划购电量的 17.901%。约束侧的复算排除两种技术性解释：全年只有 57 个时段即全部 48,096 个时段的 0.1185% 充电达到功率上限，落在这些时段的弃置仅 1,172.5 kWh，弃置总量的 99.97% 即 3,847,546.6 kWh 发生在充电未达功率上限的时段；储能全天最高 6,609.6171 kWh，距上限 4,190.3829

kWh, 从未触顶。排除这两者后, 弃置只能由多余的来源在当天无处可用且在目标函数中不被跨日赋值解释, 而后者正是跨日近视假设本身。该信号仍然不是对假设的证伪, 因为在近视模型内把无处可用的电量弃置确实是最优的; 它的作用是标出假设的代价可能有 17.61% 的计划购电量级, 从而说明扫描检验不可省略。出处 outputs/taskComposite/dispatch_year.csv 与 outputs/taskComposite/dispatch_detail.csv。

信号三：原论证的两端几乎相等与电价表的复算结果不符。晚高峰 22:00—24:00 的实际电价区间是 0.3859 至 0.4368 元/kWh, 均值 0.4215, 有 1 个时段低于原论证引用的下界 0.418; 凌晨 0:00—5:00 的电价区间是 0.4245 至 0.4422 元/kWh, 均值 0.4334, 反而高于晚高峰均值 2.81%; 全天最低价 0.3713 元/kWh 出现在 5:40, 并不在 22:00—24:00 窗口内。放掉一侧的晚高峰均价 0.4215 比留下后在凌晨使用一侧的凌晨均价 0.4334 更低, 两端不是几乎相等, 而是存在方向明确、量级约 2% 至 3% 的系统性缺口, 论证的比较没有考虑这一缺口, 也没有考虑储能往返效率 $\eta_c\eta_d = 0.81$ 对两侧的不同作用。出处 ProblemC/附件/附件 1.xlsx。

信号四：晚高峰仍在弃置, 而同一时刻日末储能恰好停在下限。晚高峰 22:00—24:00 窗口内计划购电 2,299,967.1 kWh、实际净负荷 2,285,454.0 kWh、放电 47,079.6 kWh、充电 20,182.2 kWh, 同时弃置 46,416.7 kWh。在这一窗口内储能还有 4,190 kWh 以上的容量余量, 日末储电量却精确停在下限。这是手边有容量、有富余来源却既不留也不充的直接痕迹, 与论证预期的两个模型在日末趋于一致之间的落差, 正是远视化要回答的问题。出处 outputs/taskComposite/dispatch_detail.csv。

附录 B 充分条件不可检验性的形式化否定

B.1 两个目标函数的写法

记第 d 天的近视目标为

$$\min \sum_{t \in \mathcal{T}} P_t G_{d,t} + \sum_{\omega \in \Omega_d} \pi_{d,\omega} \sum_{t \in \mathcal{T}} 5P_t E_{d,t}^\omega + \varepsilon \sum_{\omega \in \Omega_d} \pi_{d,\omega} \sum_{t \in \mathcal{T}} (C_{d,t}^\omega + D_{d,t}^\omega), \quad (36)$$

远视目标为

$$\min \sum_{t \in \mathcal{T}} P_t G_{d,t} + \sum_{\omega \in \Omega_d} \pi_{d,\omega} \left(\sum_{t \in \mathcal{T}} 5P_t E_{d,t}^\omega + \theta_d^\omega \right), \quad \theta_d^\omega \geq a_{d,q} S_{d,145}^\omega + b_{d,q}. \quad (37)$$

原论证所用的充分条件是

$$\max_{q=0,\dots,Q-1} (-a_{d,q}) \lesssim \min_{t \in \mathcal{T}^{\text{晚}}} P_t, \quad \mathcal{T}^{\text{晚}} = \{133, \dots, 144\}, \quad (38)$$

其左端 $-a_{d,q}$ 是终端价值函数 $V_{d+1}(s)$ 的切线斜率。

B.2 六步论证

第 1 步。 $a_{d,q}$ 只在远视模型中作为决策约束的系数出现。符号表把 $(a_{d,q}, b_{d,q})$ 定义为终端价值辅助变量 θ_d^ω 的第 q 条切线的斜率与截距, 而 θ_d^ω 只出现在远视模型的目标函

数与支撑约束中。

第2步。近视模型族中 θ 不进入目标函数，因此没有任何最优性条件涉及 $a_{d,q}$ 。近视模型的最优解 $(G^*, C^*, D^*, S^*, E^*, W^*)$ 与 $a_{d,q}$ 无关；既有落盘产物中不存在任何记录 $a_{d,q}$ 的文件或列。

第3步。要在近视模型族内给 $-a_{d,q}$ 赋值，唯一可能的取值是 0。若坚持在近视模型内解释 θ_d^ω ，则它不受任何下界约束，又以正权重 $\pi_{d,\omega}$ 进入最小化目标，故 $\theta_d^\omega \equiv 0$ 对一切 ω ，从而 $a_{d,q} \equiv 0$ 。

第4步。于是充分条件退化为 $0 \lesssim 0.385900$ 。右端 $\min_{t \in \mathcal{T}^{\text{晚}}} P_t = 0.385900 > 0$ ，不等式无条件成立，不依赖任何数据、情景池或权重。

第5步。该条件因此等价于近视模型不为日末储能赋值，即跨日近视假设本身。以假设为前提推出条件、再用条件支持同一假设，构成循环；条件不是对假设的检验，而是对假设的重述。

第6步。要真正计算左端，必须先构造 $V_{d+1}(s)$ ，而这已经是远视模型。 $V_{d+1}(s)$ 定义为次日以储能 s 开局的最小期望费用，其切线斜率必须由远视或至少一步前瞻的模型给出。该条件只有在远视模型已经建好之后才能被检验，它不能用来决定是否需要建远视模型。

否证的落点。原论证在当前模型族内不可检验，这是否证的结论。否证的对象不是跨日近视近似不准这一数值判断，而是该论证构成对跨日近视假设的检验这一方法论主张。这一结论只依赖模型结构，不依赖具体数据，换一批电价或负载曲线也不会改变。

B.3 使该条件可检验所需的构造

要使上述充分条件成为可判定的，需要依次完成三步：显式定义 $V_{d+1}(s)$ ，包括状态区间上的采样网格、采样点上的值函数与共同出发点约束的次梯度、以及每点一条切线的构造规则；在同一情景池与同一核权重下把切线送入第 d 天的目标函数，得到远视模型；在同一执行层下配对比较 C_{official} ，并按事先固定的相对阈值给出采纳判定。这三步构成附录 C 的主体。出处 `outputs/taskCompositeFar/assumption_audit.md` 与 `outputs/taskCompositeFar/assumption_audit.json`。

附录 C 远视模型、配对对照与核验

C.1 终端价值函数的构造

终端价值函数 $V_{d+1}(s)$ 定义为以储能 s 开局、在第 d 天已知的 $(\Omega_d, \pi_{d,\omega})$ 下单日近视问题的最小期望费用。截断口径取一步前瞻加近视尾部，即 V_{d+1} 的尾部仍取近视，不含更远的终端价值，递归不窥见未来；期望用与主模型逐元素一致的同一组 $(\Omega_d, \pi_{d,\omega})$ 估计；次梯度取共同出发点约束 $S_{d,1}^\omega = s$ 的对偶变量之和。终端价值是费用函数，关于开局储电量单调不增且凸，因此 $a_{d,q} \leq 0$ 且随 q 单调不减。由于 θ_d^ω 以正权重 $\pi_{d,\omega}$ 进入最小化目标，线性规划自动取 $\theta_d^\omega = \max_q(a_{d,q}S_{d,145}^\omega + b_{d,q})$ ，即切线的上包络。由值函数的次梯度性质，每条切线都在 V_{d+1} 下方，包络自下方支撑 V_{d+1} ，不会高估留电的价值。

全部判据、网格规则与判定阈值在读取任何远视与近视对照结果之前写入 `outputs/taskCompositeFar/frozen_rule_far.json`，其中字段 `pre_registered_before_reading_results`

标记为真。核心口径为：状态区间 $[S^{\min}, S^{\max}] = [1200, 10800]$ ；初始网格 $N_0 = 5$ ，轮数上限 $R_{\max} = 5$ ；加密方式为保留已算点的中点二分；每个采样点生成一条切线，不取区间端点中较紧者；切线指纹容差为斜率 10^{-8} 元/kWh、截距 10^{-6} 元；切线集合逐日重算，不沿用前一日的切线；冷启动期 2025-01-01 至 01-31 取 $\theta \equiv 0$ ，以保证 02-01 的日初储电量与近视模型逐元素一致。

C.2 网格加密与收敛判据

加密逐轮记录如表 11，为便于排版，该表以轮次为列、指标为行给出；出处 `outputs/taskCompositeFar/value_function_convergence.md` 与 `outputs/taskCompositeFar/value_function_rounds.csv`。

表 11 终端价值函数采样网格的逐轮加密记录

指标	第 0 轮	第 1 轮	第 2 轮	第 3 轮	第 4 轮	第 5 轮
网格点数 N_r	5	9	17	33	65	129
网格间距 (kWh)	2,400	1,200	600	300	150	75
到达该轮的天数	334	334	331	331	331	4
最大相邻差 (元)	1,148.52	574.8	287.4	143.7	71.8502	35.4001
中位相邻差 (元)	876.359	539.39	282.126	141.5	70.7545	35.4001
包络抬高最大值 (元)	-1	123.681	84.1386	40.4151	23.6457	0.329385
新增有效切线 (严格集合差)	-1	4	8	16	32	11
有效切线数 (最大)	5	9	17	33	65	34
在该轮判收敛的天数	0	3	0	0	327	4

第 0 轮的包络抬高量与新增有效切线记为 -1，因为该轮没有上一轮可比，不是数值为零。第 $r \geq 1$ 轮的两行分别是第 r 轮相对第 $r-1$ 轮的包络抬高量 $\max_s [\hat{V}_r(s) - \hat{V}_{r-1}(s)]$ 与严格集合差计数。全年 334 天全部判收敛，实际用到的最多轮数为 6，最大网格点数为 129，收敛轮次的分布为第 1 轮 3 天、第 4 轮 327 天、第 5 轮 4 天，累计求解决策层线性规划 21,798 次，求解器累计耗时 62,451.0 秒，为并行前的单核口径。最终切线集合的每日条数在 5 至 65 条之间，总计 15,612 条，存放于 `outputs/taskCompositeFar/value_function_samples.npz` 的 `tangents` 数组。

两条判据的定义。判据 A 要求相邻网格点上价值函数估计值之差的最大值不超过 $\eta = 150$ 元；判据 B 要求再加密一轮的包络抬高量不超过 $\tau_{\text{env}} = 15$ 元。判据 B 的原始表述是再加密一轮不再产生新的有效切线，对严格凸的价值函数，新增采样点的切线一定在该点接触 V ，因此按集合论意义的严格集合差恒不为零，本批数据中各轮的最大严格新增数为第 1 轮 4 条、第 2 轮 8 条、第 3 轮 16 条、第 4 轮 32 条、第 5 轮 11 条。若照字面执行判据永不成立，只能撞上轮数上限，这会掩盖真实情况。因此把新操作化为该轮新切线是否在容差之外抬高了逼近包络，该量有直接的经济含义，即把网格再加密一轮能给价值函数逼近带来的最大改进。严格的集合差仍逐日如实落盘在 `value_function_rounds.csv` 的 `new_effective_tangents` 列，未被掩盖。

阈值的标定依据。判据 A 的量纲是相邻网格点上的费用差，把它按日累加得到全年上界 $334 \times \eta = 334 \times 150 = 50,100$ 元，占既有近视结果 $C_{\text{official}} = 16,998,506$ 元的 0.295%，低于事先固定的采纳阈值 0.5% 即 84,993 元。即便判据 A 的分辨率极限全部转化为同向误差，也不足以把一个达到采纳标准的改善掩盖掉。判据 B 同理， $334 \times 15 = 5,010$

元即 0.029%。阈值不是从对照结果反推的，而是由 `value_function.py --calibrate` 在 12 个代表日上跑满轮数上限得到的轨迹确定，记录于 `outputs/taskCompositeFar/value_function_calibration.json`。标定轨迹中第 3 轮的 $\max \Delta = 142.3003$ 元、第 4 轮的包络抬高量最大值 = 11.8174 元，因此 $\eta = 150$ 、 $\tau_{\text{env}} = 15$ 使网格在第 4 轮即 65 点判定收敛，而更严的取值如 $\eta = 100$ 需要第 5 轮即 129 点才能满足判据 A。标定过程只读取近视模型的值函数，不接触任何远视与近视对照结果，对照结果在该文件生成之后才产生。

逼近质量。采样点处逼近精确， $\hat{V}_r(s_k) = v_k$ ，逐日的 `max_envelope_deficit_yuan` 均不大于 0；次梯度符号与单调性方面，`max_slope_positive_violation` 与 `max_slope_monotone_violation` 的全年最大值分别为 8.973×10^{-7} 与 1.408×10^{-10} 元/kWh，均在 10^{-6} 容差内； $s = S^{\max}$ 处的上界约束绑定，其次梯度是单侧的，`boundary_slope_jump` 单独记录，不计入单调性判据。

C.3 含终端价值项的数值处理与求解器逐级回退

加入 θ 后模型的数值条件明显变差： θ 是自由变量、量级数万元， $a_q \approx -0.5$ 、 $b_q \approx 5 \times 10^4$ ，与量级 10^3 kWh 的能量平衡约束不同尺度。HiGHS 的默认 `presolve` 在 2025-03-25 返回 status 15，即 `model_status Unknown`、`primal_status Infeasible`；而关闭 `presolve` 后在 2025-02-11 返回同样的状态。两种设置各有失败个案，但模型在这两天都可行，前者在 `presolve=False` 下成功，后者在默认设置下成功。

因此求解器入口按事先固定的顺序逐级回退：`highs+presolve` 与基线逐元素相同、绝大多数日走这一级，随后依次为 `highs-no-presolve`、`highs-ds-no-presolve`、`highs-ipm`。实际走过的级别逐日记录在 `outputs/taskCompositeFar/dispatch_year_far.csv` 的 `solver_path` 列，全年分布为 `highs+presolve` 332 天、`highs-no-presolve` 2 天。无论走哪一级，解都要通过 14 条逐日残差断言，因此求解路径不改变模型定义与结论。价值函数采样与近视基线仍使用原设置，两个模型的可比性不受影响。

C.4 配对对照的完整数据

两个模型使用同一情景池、同一核权重、同一执行层即逐元素相同的补救问题装配、同一冷启动口径，因此逐日差值的唯一来源是目标函数中的终端价值项 $\sum_{\omega \in \Omega_d} \pi_{d,\omega} \theta_d^\omega$ 。2025-01-01 至 01-31 的冷启动期两模型都取 $\theta \equiv 0$ ，所以 02-01 的日初储电量逐元素相同，不存在初始条件差异。逐日明细见 `outputs/taskCompositeFar/comparison_daily_far_vs_near.csv`，汇总见 `outputs/taskCompositeFar/comparison_far_vs_near.md`。

采纳判定见表 12。

判定规则在读取任何对照结果之前写入 `outputs/taskCompositeFar/frozen_rule_far.json` 的 `adoption_rule` 字段：相对节省不低于 0.50% 才采纳远视，否则保留近视并如实记录负结果；阈值不因结果回调，评分口径沿用 C_{official} 的定义且分母取远视费用。

逐日配对结构见表 13。

符号检验的 p 值仅作描述性报告，不参与判定，因为配对口径下的逐日差值并非独立同分布样本。

结果结构对比如表 14。

表 12 远视模型相对近视基线的采纳判定

项目	数值
配对天数	334
近视 C_{official} (元)	16,998,505.69
远视 C_{official} (元)	17,036,556.63
相对节省 $(C_{\text{near}} - C_{\text{far}})/C_{\text{far}}$	-0.2233%
绝对节省 (元)	-38,050.94
事先固定的采纳阈值	0.50%
判定	保留近视模型

表 13 逐日配对差值结构

项目	数值
逐日差值均值 (元)	-113.92
逐日差值中位数 (元)	-0.23
逐日差值标准差 (元)	438.50
远视更省的天数	23
近视更省的天数	178
差值绝对值小于 10^{-9} 元的天数	133
单日最大节省 (元)	4,811.50
单日最大亏损 (元)	-1,621.24
符号检验双侧 p 值	7.015×10^{-31}

费用构成对比。近视侧计划购电费 16,814,531.4 元、紧急购电费 183,974.3 元；远视侧计划购电费 16,869,430.5 元、紧急购电费 167,126.1 元；两模型合计分别为 16,998,505.7 元与 17,036,556.6 元。出处 outputs/taskCompositeFar/validation_report_far.md 第五节。

C.5 负结果的传导机制分析

终端价值项确实起了作用，但作用被执行层吸收。决策层的情景层日末储电量从近视模型的恒定 1200 kWh 提升到全年加权均值 6,398.5 kWh，全年范围为 1,264.3 至 10,763.6 kWh，说明模型愿意为次日留电。出处 outputs/taskCompositeFar/dispatch_year_far.csv 的 dec_storage_end_mean、dec_storage_end_min、dec_storage_end_max 列。

决策层的这一改变在全年总量上表现为：计划购电量上升 128,863.1 kWh，相对上升 0.59%；弃置电量上升 126,047.0 kWh，相对上升 3.3%；紧急购电量减少 5,709.6 kWh，相对减少 10.2%。终端价值项抬高了报童式计划的临界分位数，因为过剩成本由 P_t 降到 P_t 减去留电的边际价值，紧急购电的收益不足以覆盖计划购电费的上升。传导链条

表 14 近视模型与远视模型的结果结构对比

项目	近视	远视
计划购电量 (kWh)	21,850,664.3	21,979,527.4
紧急购电量 (kWh)	55,844.1	50,134.5
弃置电量 (kWh)	3,848,719.1	3,974,766.0
储能充电量 (kWh)	624,209.4	608,981.0
储能放电量 (kWh)	505,609.6	493,274.6
日末储电量均值 (kWh)	1,200.0000	1,200.0000
日末储电量最小值 (kWh)	1,200.0000	1,200.0000
日末储电量最大值 (kWh)	1,200.0000	1,200.0000
C_{official} (元)	16,998,505.69	17,036,556.63

被执行层切断的原因在于执行层逐元素沿用定稿的补救问题装配，不含终端价值项，它在真实曲线上重新优化时仍把日末储电量压回下限，于是留电到次日这条通道被执行层切断， C_{official} 只剩下计划购电量的微小变化。

附加的单日诊断。附加诊断每月取 1 天、共 11 天，每天独立求解，日初储电量取实际链上的值，不重新滚动，并把同一组终端价值切线也放进执行层的目标函数。结果见表 15。

表 15 执行层加入终端价值切线的单日诊断

项目	近视执行层	远视执行层
日末储电量均值 (kWh)	1,200.00	6,173.59
日末储电量范围 (kWh)	—	1,200.00—10,799.77
紧急购电量合计 (kWh)	12,068.69	12,068.69
当日费用合计 (元)	562,638.52	562,638.52

把同一组切线也放进执行层后，日末储电量均值升到 6,173.59 kWh，而当天费用合计逐日完全相同，说明留电的价值只在次日体现，单日诊断看不到，这也是本工作未能补做完整滚动对照的地方。逐日明细见 `outputs/taskCompositeFar/execution_lookahead_probe.csv`。

这个负结果能说什么、不能说什么。它回答的是在既定执行层下把终端价值补进决策层能否改善官方结算费用，答案是不能。它不能回答跨日近视究竟有多大代价：执行层的近视性是配对口径的一部分，不是本次对照的发现；要回答后一个问题，必须让执行层也为次日留电并重新滚动整条决策层链，那已超出本步骤声明的计算预算，如实留作未完成的检验。它同样不能说明跨日近视假设已被证明正确，因为本次对照只在决策层远视而执行层近视这一具体口径下给出了一个负结果，而原论证的不可检验性结论与数据无关，依然成立。

适用边界的四条限定。其一，截断口径。终端价值取一步前瞻加近视尾部，不是无限期值函数；若把尾部也换成远视， V_{d+1} 的水平会下降，因为多阶段最优不高于单阶段最优，留电的边际激励随之改变，因此本对照给出的差值是在尾部仍近视这一口径下的差值。其二，执行层仍是近视的，远视化的全部作用都通过决策层的计划购电量传导，执行层的实际充放电路径仍按当日的紧急购电费用最小化排序。其三，冷启动期。2025-01-01 至 01-31 两模型都取 $\theta \equiv 0$ ，这是为保证 02-01 日初储能一致而事先固定的口径，不是模型结论。其四，年末截断。12-31 的终端价值对应 2026-01-01 的假想次日，其费用不计入 C_{official} ，该口径在滚动窗口假设下与其余各日一致。

C.6 远视模型的核验结果

远视模型在近视模型的 12 条断言之外新增 9 条集合与逐日断言，合计 21 条。逐日断言 C1 至 C14 全年 365 天全部通过，集合级断言 C15 至 C21 全部通过。汇总见 `outputs/taskCompositeFar/validation_report_far.md`，逐日记录见 `outputs/taskCompositeFar/validation_daily_far.csv`。逐日断言的最坏值见表 16。

C13 与 C14 的容差取混合形式 $\max(10^{-4} \text{ 元}, 10^{-6} \times \max(1, \max_{\omega} |\theta_{\omega}|))$ 。 θ 的量级是数万元，用纯绝对容差会把求解器可行性容差放大后的正常数值误判为违规，用纯相对容差又会在 θ 小时过严；绝对残差与相对残差在报告中同时给出。以 $\theta \sim 5 \times 10^4$ 元计，允许的绝对残差约 5×10^{-2} 元/天，比终端价值本身的估计精度即 $\tau_{\text{env}} = 15$ 元小三个数

表 16 远视模型逐日断言 C1 至 C14 的全年最坏值

编号	断言	容差	全年最坏值
C1	能量平衡 $\max_t G_t + E_t + R_t + D_t - L_t - C_t - W_t $	10^{-6}	2.274e-13
C2	储能递推 $\max_t S_{t+1} - S_t - \eta_c C_t + D_t/\eta_d $	10^{-6}	4.690e-13
C3	日初状态 $ S_1 - S_{\text{start}} $	10^{-6}	0.000e+00
C4	储能边界越界量	10^{-6}	0.000e+00
C5	充放电功率越界量	10^{-6}	0.000e+00
C6	非负性违反量	10^{-6}	1.137e-13
C7	同时充放电时段数	0	0
C8	紧急购电与充电同时为正的时段数	0	0
C9	跨日链残差	10^{-6}	0.000e+00
C10	费用重算相对误差	10^{-9}	0.000e+00
C11	充电来源违反量	10^{-6}	0.000e+00
C12	紧急购电上界违反量	10^{-6}	0.000e+00
C13	终端价值自下方支撑 $\max_{\omega, q} [a_q S_{\omega, 145} + b_q - \theta_\omega]$	混合	绝对 1.597e-04 元, 相对 2.933e-09
C14	终端价值取到切线上包络 $\max_\omega [\theta_\omega - \max_q a_q S_{\omega, 145} - b_q]$	混合	绝对 1.209e-07 元, 相对 2.142e-12

量级。C13 与 C14 只在当天有终端价值切线的事情上有定义，2025-01-01 至 01-08 的候选历史池为空或不足，按冷启动口径当日计划购电量取零且 $\theta \equiv 0$ ，这两条断言记为缺失，全年实际测量 357 天、缺失 8 天，最坏值只在实际测量的天上取。集合级断言的结果见表 17。

表 17 远视模型集合级断言 C15 至 C21 的结果

编号	断言	结果
C15	切线斜率非正 $\max_q a_q \leq 10^{-6}$	全年最大正值 8.973e-07 元/kWh
C16	切线斜率随 q 单调不减	全年最大违反 0.000e+00 元/kWh
C17	值函数关于 s 单调不增	全年最大违反 8.095e-08 元
C18	次梯度单调不减（内部点）	全年最大违反 9.754e-11 元/kWh
C19	网格收敛（判据 A 与判据 B）	收敛 334/334 天
C20	$\theta \equiv 0$ 退化	比较 334 天，不一致 0 天，最大差 4.992e-07 kWh，通过
C21	因果性（2025-06-21 扰动）	切线集合一致、计划一致，通过

C20 的分层核实验。装配层把 θ 完全不添加时，远视装配与近视装配的 c 、 A_{eq} 、 b_{eq} 、 A_{ub} 、 b_{ub} 、变量数与边界数逐元素相同，全年最大差异 0.000e+00，说明远视装配没有引入任何额外偏差。求解层把逐日终端价值切线全部置零后重跑决策层，比较 334 天，计划购电量最大绝对差 4.992×10^{-7} kWh、相对差 4.864×10^{-10} 、 $\max |\theta| = 0.000e+00$ 元。严格逐元素相同在浮点求解器下不可得，因此以 10^{-6} kWh 即与残差容差同量级为判据，并同时报告相对量，未掩盖任何一项。出处 `outputs/taskCompositeFar/degeneracy_test.json`。

C21 目标日扰动因果性检验。选 2025-06-21，把该日及其之后的全部原始负载与光伏观测打乱并施加 $1.37x + 31$ 的扰动，重建处境特征后重算该日的候选池、核权重与终端价值切线集合，再装配远视模型求解。结果为候选池完全一致、核权重逐元素一致、终端价值切线集合逐元素一致即 61 条对 61 条、计划购电量逐元素一致且最大绝对变化 0 kWh、目标日观测确实已被改动、该日之前的处境特征逐元素不变。这证明第 d 天的切线集合只依赖 $j < d$ 的观测，远视递归没有引入前视污染。需要注意这一点无法由残差核实验发现，因为用未来数据估计 V_{d+1} 得到的仍是可行解、残差仍为零，只有扰动实验能发现。出处 `outputs/taskCompositeFar/causality_test.json`。

决策层情景层残留。决策层的情景层核验另有 341 天出现少量残留，逐日计数见 `outputs/taskCompositeFar/decision_layer_residual_far.json`；这一数字与近视基线同为 341 天，说明终端价值项没有引入新的情景层残留。这些残留只存在于情景设想层，不影响任何填报数值。残留的性质是第二阶段在完美信息下，对缺口大于期望缺口的情景仍选择先充电的边界情形；执行层已用 C12 的常数上界彻底排除该行为。

结果结构核对。`result2_far.xlsx` 三张工作表的表头与附件 5 模板逐字一致，计划购电量 335 行 $\times$ 147 列且 334 天填满无缺失，充放电量 2005 行 $\times$ 6 列，紧急购电量 338 条非零记录。该文件与 `outputs/taskComposite/result2.xlsx` 的工作表行列结构一致，指定四日表格两两交叉一致，见 `outputs/taskCompositeFar/tables_far.md`。按采纳判定，正文与最终填报仍以近视模型为准。

规模与预算。终端价值采样求解 21,798 次，单核累计耗时 62,451.0 秒；远视主求解 334 天；退化核验 334 天；决策层单日变量规模为 $144 \times (2 + 5|\Omega_d|) + |\Omega_d|$ ；终端价值约束条数为 $|\Omega_d| \times Q_d$ ， Q_d 为当日有效切线数。

附录 D 赋权方案设计的完整证据

D.1 特征消融的完整表格

候选顺序在读取任何消融结果之前固定为 `net_lag_7d`、`weekday7`、`load_mean_3d`、`pv_mean_3d`、`weekend`、`load_lag_7d`、`pv_lag_7d`、`net_mean_7d`、`net_mean_3d`、`load_std_7d`、`pv_std_7d`；`season` 作为日历结构先验置于最前，是一月窗口无法识别的结构性例外，不占四个语义特征名额。消融固定 $h = 1$ ，纳入条件为平均能量分数相对改善超过 0.5% 且至少 60% 的验证日严格改善。完整数值见表 18，出处 `outputs/taskComposite/feature_ablation.csv` 与 `outputs/taskComposite/feature_report.md`。

表 18 按预先固定顺序逐步纳入特征的消融结果

依序候选	加入前平均 ES(kW)	加入后平均 ES(kW)	相对改善	改善日数	判定
<code>net_lag_7d</code>	1324.825	741.507	+44.030%	320/334	纳入
<code>weekday7</code>	741.507	726.260	+2.056%	144/334	不纳入
<code>load_mean_3d</code>	741.507	735.862	+0.761%	164/334	不纳入
<code>pv_mean_3d</code>	741.507	620.378	+16.336%	291/334	纳入
<code>weekend</code>	620.378	625.759	-0.867%	171/334	不纳入
<code>load_lag_7d</code>	620.378	556.874	+10.236%	246/334	纳入
<code>pv_lag_7d</code>	556.874	555.915	+0.172%	222/334	不纳入
<code>net_mean_7d</code>	556.874	553.097	+0.678%	232/334	纳入
<code>net_mean_3d</code>	553.097	570.830	-3.206%	114/334	不纳入
<code>load_std_7d</code>	553.097	608.354	-9.990%	141/334	不纳入
<code>pv_std_7d</code>	553.097	565.119	-2.174%	135/334	不纳入

固定顺序的逐步纳入存在路径依赖，本工作不重排候选以追求更好的数字，失败特征的结果一并保留在 `outputs/taskComposite/feature_ablation.csv` 与 `outputs/taskComposite/feature_ablation_daily.csv`。

D.2 事后补充候选的通过过程

结构探索发现低负载日落在周五与周六，而 **weekend** 只把周六与周日标为同质，据此另提两个候选，用完全相同的判据复核。二者是在看到结构证据之后才提出的，属于事后假设，因此基础集固定为选择窗选出的特征集合，并同时确认窗上独立复核。完整数值见表 19，出处 `outputs/taskComposite/supplementary_ablation.csv` 与 `outputs/taskComposite/supplementary_ablation_confirm.csv`。

表 19 事后补充候选在两个窗口上的复核结果

窗口	补充候选	加入前平均 ES(kW)	加入后平均 ES(kW)	相对改善	改善日数	判定
全年 334 天	low_load_day	620.378	584.612	+5.765%	263/334	通过
全年 334 天	weekday7	584.612	577.189	+1.270%	156/334	不通过
确认窗 184 天	low_load_day	607.224	566.802	+6.657%	159/184	通过
确认窗 184 天	weekday7	566.802	559.450	+1.297%	92/184	不通过

两个窗口都通过的事后候选为 **low_load_day**。它的编码来自纯描述性的星期几结构证据，即低负载日落在周五与周六，与能量分数无关，并在时间上独立的确认窗复现了同向改善，故并入最终冻结规则；方法上仍标注为事后补充，证据强度低于按预先固定顺序纳入的主消融特征。

D.3 带宽搜索的完整过程

修订特征集上重新搜索带宽，先在 $[0.05, 20]$ 上作 25 点对数粗搜，再作三轮各 17 点细搜，同时记录等权极限，共 77 次候选评估，结果 $h = 1.043841464$ ，状态为 **interior_optimum**。旧特征集在同一协议下独立搜索带宽得 $h = 0.390601653$ ，两者带宽各自冻结，不交叉调参。仅平均能量分数参与选优，有效情景数只作诊断。搜索轨迹见 `outputs/taskComposite/bandwidth_search_final.csv` 记录修订特征集、`outputs/taskComposite/bandwidth_search_legacy.csv` 记录旧特征集，每份均为 77 行候选记录，其中 **uniform_limit** 一行对应等权极限 $h = \infty$ ，其平均能量分数为 1334.485697 kW。

D.4 新旧赋权方案的全部对比

评分对比。等权基线平均能量分数 1334.485697 kW，旧特征集 1248.040416 kW，修订特征集 551.071402 kW。修订相对等权改善 58.705335%，改善日数 328/334；相对旧特征集改善 55.845068%，改善日数 318/334。采纳门槛为 3.0%，判定为采纳修订特征集。出处 `outputs/taskComposite/frozen_rule.json`。

样本外确认。验证期按时间一分为二，选择窗 2025-02-01 至 2025-06-30 共 150 天，确认窗 2025-07-01 至 2025-12-31 共 184 天，两窗的池、标尺与评分规则完全一致。完整数值见表 20，出处 `outputs/taskComposite/split_confirmation.csv`。

有效情景数对比。修订特征集的 K_{eff} 均值 12.17、范围 1.01 至 40.08；旧特征集均值 12.98、范围 1.23 至 26.70；等权基线均值等于池规模 46.685629、范围 24 至 84。周同期特征让权重更集中，这是平均能量分数改善的主要来源，同时意味着情景覆盖变窄，因此另做决策层抽样回测兜底。

表 20 三套赋权方案在两个时间窗上的样本外确认

窗口	方法	天数	平均 ES(kW)	相对等权
选择窗	等权基线	150	1370.781	+0.000%
选择窗	旧特征集	150	1263.269	+7.843%
选择窗	修订特征集	150	582.713	+57.490%
确认窗	等权基线	184	1304.897	+0.000%
确认窗	旧特征集	184	1235.944	+5.284%
确认窗	修订特征集	184	528.936	+59.465%

决策层抽样回测。每月取前 2 天共 22 天，用三套权重分别求解决策层与执行层，在真实曲线上结算，日初储电量统一取 6000 kWh 以隔离权重差异。完整数值见表 21，出处 `outputs/taskComposite/backtest_sampled.csv`。

表 21 三套赋权方案在 22 天抽样上的真实曲线回测

方法	天数	计划购电总量 (kWh)	紧急购电总量 (kWh)	总费用 (元)	平均 K_{eff}
修订特征集	22	1,394,727.4	14,065.2	1,125,932.8	10.84
旧特征集	22	1,515,486.7	20,030.7	1,203,114.1	10.69
等权基线	22	1,491,144.3	39,212.5	1,255,970.2	32.41

修订特征集总费用 1,125,932.8 元，比等权基线 1,255,970.2 元低 10.354%。能量分数上的改善确实转化为真实费用与紧急购电量的下降。

冻结规则的最终内容。最终特征集合为 `season、net_lag_7d、pv_mean_3d、load_lag_7d、net_mean_7d、low_load_day`，带宽 $h = 1.0438414641946645$ ，季节月份环形距离 1，预热 7 日，标尺随池滚动。规则自 2025-02-01 起可用。完整记录见 `outputs/taskComposite/frozen_rule.json`。

D.5 附带记录的两项口径说明

时间标签口径。附件 5 的 `result2` 模板首段标签为 0:10-0:20、末段标签为 0:00-0:10+1，比附件 2 的右端点标签整体前移一个时段。主口径采用位置对齐，即附件第 t 个观测对应模板第 t 个数据列，正文表 1 即按此口径填报。作为对照，字面时钟口径下四日的六个指定时段数值分别为 2025-03-20 的 33.15、0.00、27.55、514.51、688.24、718.41，2025-06-21 的 1.83、0.00、7.19、169.77、473.16、638.07，2025-09-23 的 0.50、-0.00、1.51、478.16、773.96、816.72，2025-12-21 的 339.13、119.47、314.44、808.63、705.48、726.84。两种口径的全天合计相同，差异只体现在逐时段错位上。出处 `outputs/taskComposite/validation_report.md` 第三节。

结果文件结构。`result2.xlsx` 三张工作表的表头与附件 5 模板逐字一致；计划购电量工作表 335 行 $\times$ 147 列，334 天填满无缺失；充放电工作表 2005 行 $\times$ 6 列，每天 6 个四小时时段，0:00 与 24:00 储电量填在当日前两行的 E、F 列；紧急购电量工作表 362 行 $\times$ 3 列，共 361 条非零记录，按模板表 4 示例合并连续时段且只填非零。模板只给出前三天的格式示例，因此后两张表按实际记录展开行数，列结构与标签完全照抄模板。完整结构记录见 `outputs/taskComposite/result2_structure.json`。

程序代码

本节给出问题一与问题二核心求解代码的完整清单。源码文件同时保存在支撑材料的 code 目录中，与论文中的模型表述逐条对应：问题一的主线性规划见 solve_q1.py，问题二的模型装配与求解见 model.py，两阶段调度的规划层与执行层见 dispatch.py。

问题一主求解脚本 solve_q1.py

该脚本读取同目录的 附件 1.csv，在日循环边界下求解问题一的主线性规划，输出逐时段调度明细、中文汇总与结果填报表。

代码 1 问题一主求解脚本 solve_q1.py

```
1 """问题1：自由初始储电量、日循环边界下的主线性规划。
2
3 运行：python -B solve_q1.py
4 输入和输出均相对于本脚本定位；主解不添加互斥或次级目标。
5 另外分析最优费用下的初始电量区间。
6 仅输出完整调度dispatch.csv和中文汇总summary.json，不生成中间或写作文件。
7 """
8
9 import csv
10 import hashlib
11 import io
12 import json
13 import platform
14 from pathlib import Path
15
16 import numpy as np
17 import scipy
18 from scipy.optimize import linprog
19 from scipy.sparse import csr_matrix, lil_matrix, vstack
20
21
22 N = 144
23 DELTA = 1 / 6
24 S_MIN, S_MAX = 1200.0, 10800.0
25 MAX_ENERGY = 5000 * DELTA
26 ETA_C = ETA_D = 0.9
27 TOL = 1e-6 # 电量、约束残差的绝对容差，单位kWh。
28 SOLVER_OPTIONS = {"primal_feasibility_tolerance": 1e-8,
29                  "dual_feasibility_tolerance": 1e-8}
30
31
32 def time_label(boundary):
33     """boundary=0和144分别表示当天00:00和24:00。"""
34     minutes = boundary * 10
35     return f"{minutes // 60:02d}:{minutes % 60:02d}"
36
37
38 def load_data(path):
39     raw = path.read_bytes()
```

```

40 reader = csv.DictReader(io.StringIO(raw.decode("utf-8-sig")))
41 columns = ["时段序号", "时间", "电价", "小区负载", "光伏发电预测功率",
42           "负载能量(kWh)", "光伏能量(kWh)"]
43 if reader.fieldnames != columns:
44     raise ValueError(f"CSV列名或顺序不符: {reader.fieldnames}")
45 rows = list(reader)
46 if len(rows) != N:
47     raise ValueError(f"应有{N}个时段, 实际为{len(rows)}")
48 for t, row in enumerate(rows, start=1):
49     if None in row or any(value is None or not value.strip() for value in row.values()):
50         raise ValueError(f"第{t}段存在缺失值或多余字段")
51     allowed_times = {time_label(t)} if t < N else {"0:00+1", "00:00+1", "24:00"}
52     if int(row["时段序号"]) != t or row["时间"].strip() not in allowed_times:
53         raise ValueError(f"第{t}段序号或时间标签不符")
54
55 values = np.array([[float(row[name]) for name in columns[2:]] for row in rows])
56 if not np.isfinite(values).all() or (values < 0).any():
57     raise ValueError("数值列包含非有限值或负值")
58 price, load_power, pv_power, stored_L, stored_R = values.T
59 if (price <= 0).any():
60     raise ValueError("当前问题1假设电价严格为正")
61
62 # 功率乘时长得到电量; CSV已有电量列只核验, 不重复换算。
63 L, R = load_power * DELTA, pv_power * DELTA
64 errors = {"load_energy_max_difference_kwh": float(np.max(np.abs(L - stored_L))),
65          "pv_energy_max_difference_kwh": float(np.max(np.abs(R - stored_R)))}
66 if max(errors.values()) > TOL:
67     raise ValueError(f"功率与电量列不一致: {errors}")
68 audit = {"filename": path.name, "sha256": hashlib.sha256(raw).hexdigest(),
69          "period_count": N, "energy_source": "power_columns_times_1_over_6",
70          "price_min": float(price.min()), "price_max": float(price.max()),
71          "load_total_kwh": float(L.sum()), "pv_total_kwh": float(R.sum()),
72          **errors}
73 return price, L, R, audit
74
75
76 def build_lp(price, L, R):
77     """变量顺序为[G(144), C(144), D(144), W(144), S(145)]。"""
78     for name, vector in (("price", price), ("L", L), ("R", R)):
79         if np.shape(vector) != (N,) or not np.isfinite(vector).all():
80             raise ValueError(f"{name}必须为长度{N}的有限数值向量")
81     if (price <= 0).any() or (L < 0).any() or (R < 0).any():
82         raise ValueError("要求price>0且L、R非负")
83     objective = np.zeros(5 * N + 1)
84     objective[:N] = price
85     A_eq = lil_matrix((2 * N + 1, 5 * N + 1))
86     b_eq = np.zeros(2 * N + 1)
87     for t in range(N):
88         # 能量平衡:  $G - C + D - W = L - R$ 。
89         A_eq[t, [t, N + t, 2 * N + t, 3 * N + t]] = [1, -1, 1, -1]
90         b_eq[t] = L[t] - R[t]
91         # C、D均为母线侧电量; S[0]对应论文S_1。
92         A_eq[N + t, [N + t, 2 * N + t, 4 * N + t, 4 * N + t + 1]] = [
93             -ETA_C, 1 / ETA_D, -1, 1]

```

```

94     A_eq[-1, 4 * N] = -1
95     A_eq[-1, 5 * N] = 1 # 日循环; 不额外固定S_1为6000。
96     bounds = [(0, None)] * N + [(0, MAX_ENERGY)] * (2 * N)
97             + [(0, float(value)) for value in R] + [(S_MIN, S_MAX)] * (N + 1))
98     return objective, A_eq.tocsr(), b_eq, bounds
99
100
101 def solve_model(model):
102     objective, A_eq, b_eq, bounds = model
103     result = linprog(objective, A_eq=A_eq, b_eq=b_eq, bounds=bounds,
104                     method="highs", options=SOLVER_OPTIONS)
105     if not result.success or result.status != 0:
106         raise RuntimeError(f"LP未达到最优状态: {result.message}")
107     return result
108
109
110 def unpack(x):
111     return x[:N], x[N:2*N], x[2*N:3*N], x[3*N:4*N], x[4*N:]
112
113
114 def validate_solution(result, price, L, R, model):
115     """直接从物理公式重算残差, 不仅依赖求解器的可行性判断。"""
116     G, C, D, W, S = unpack(result.x)
117     if not np.isfinite(result.x).all():
118         raise RuntimeError("求解结果包含非有限值")
119     residuals = {
120         "energy_balance_max_abs_kwh": float(np.max(np.abs(G + R + D - L - C - W))),
121         "storage_dynamics_max_abs_kwh": float(np.max(np.abs(np.diff(S) - ETA_C*C + D/ETA_D))),
122         "daily_cycle_abs_kwh": float(abs(S[-1] - S[0])),
123         "bound_max_violation_kwh": float(max(0, -G.min(), -C.min(), -D.min(), -W.min(),
124             C.max()-MAX_ENERGY, D.max()-MAX_ENERGY, (W-R).max(), S_MIN-S.min(), S.max()-S_MAX))
125     },
126     {
127         "daily_charge_discharge_abs_kwh": float(abs(D.sum() - ETA_C*ETA_D*C.sum())),
128         "daily_energy_ledger_abs_kwh": float(abs(G.sum() - (
129             L.sum()-R.sum()+W.sum()+(1-ETA_C*ETA_D)*C.sum()))),
130     }
131     objective, A_eq, b_eq, bounds = model
132     recomputed_cost = float(price @ G)
133     cost_error = abs(recomputed_cost - result.fun)
134     # 对偶目标与原目标应一致; 独立记录最优性证据。
135     dual_value = float(b_eq @ result.eqlin.marginals)
136     for i, (lower, upper) in enumerate(bounds):
137         dual_value += lower * result.lower.marginals[i]
138         if upper is not None:
139             dual_value += upper * result.upper.marginals[i]
140     dual_gap = abs(recomputed_cost - dual_value)
141     stationarity = float(np.max(np.abs(objective - A_eq.T @ result.eqlin.marginals
142         - result.lower.marginals - result.upper.marginals)))
143     cost_tol = max(1e-6, 1e-9 * abs(recomputed_cost))
144     passed = (max(residuals.values()) <= TOL and cost_error <= cost_tol
145         and dual_gap <= cost_tol and stationarity <= TOL)
146     overlap = np.flatnonzero((C > TOL) & (D > TOL))
147     validation = {
148         "lp_checks_passed": bool(passed), "energy_tolerance_kwh": TOL,

```

```

147     "cost_tolerance_yuan": cost_tol, **residuals,
148     "objective_recalculation_abs_yuan": float(cost_error),
149     "dual_objective_yuan": dual_value, "primal_dual_gap_abs_yuan": float(dual_gap),
150     "dual_stationarity_max_abs": stationarity,
151     "mutual_exclusivity_passed": bool(len(overlap) == 0),
152     "simultaneous_charge_discharge_count": int(len(overlap)),
153     "simultaneous_charge_discharge_periods": [
154         {"t": int(t+1), "start": time_label(t), "end": time_label(t+1),
155          "C_kwh": float(C[t]), "D_kwh": float(D[t])} for t in overlap],
156     }
157     if not passed:
158         raise RuntimeError("结果校验失败: " + json.dumps(validation, ensure_ascii=False))
159     return validation
160
161
162 def analyze_solution(result, price, L, R):
163     """只统计现有主解；无储能对照直接计算，不增加优化目标。"""
164     G, C, D, W, S = unpack(result.x)
165     baseline_G = np.maximum(L - R, 0)
166     baseline_cost = float(price @ baseline_G)
167     cost = float(price @ G)
168     # 电源在母线上不可区分；以下按光伏优先供负载、富余再充电分摊。
169     pv_charge = np.minimum(C, np.maximum(R - L, 0))
170     grid_charge = C - pv_charge
171     blocks = []
172     for start in range(0, N, 24):
173         end = start + 24
174         section = slice(start, end)
175         blocks.append({
176             "时间段": f"{time_label(start)}—{time_label(end)}",
177             "购电量(kWh)": float(G[section].sum()),
178             "充电量(kWh)": float(C[section].sum()),
179             "放电量(kWh)": float(D[section].sum()),
180             "段首储电量(kWh)": float(S[start]),
181             "段末储电量(kWh)": float(S[end]),
182             "购电费用(元)": float(price[section] @ G[section]),
183         })
184     return {
185         "无储能对照": {
186             "说明": "同一负载、光伏和电价下，不使用储能，缺口购电、光伏富余弃置。",
187             "购电费用(元)": baseline_cost,
188             "购电量(kWh)": float(baseline_G.sum()),
189             "弃光量(kWh)": float(np.maximum(R - L, 0).sum()),
190             "主模型节省费用(元)": baseline_cost - cost,
191             "主模型节费率(%)": 100 * (baseline_cost - cost) / baseline_cost if baseline_cost > 0
192         }
193         else None,
194         "主模型减少购电量(kWh)": float(baseline_G.sum() - G.sum()),
195     },
196     "运行特征": {
197         "充电时段数": int(np.count_nonzero(C > TOL)),
198         "放电时段数": int(np.count_nonzero(D > TOL)),
199         "静置时段数": int(np.count_nonzero((C <= TOL) & (D <= TOL))),
200         "充电达到功率上限的时段数": int(np.count_nonzero(np.abs(C - MAX_ENERGY) <= TOL)),
201         "放电达到功率上限的时段数": int(np.count_nonzero(np.abs(D - MAX_ENERGY) <= TOL)),

```

```

200         "外网最大购电功率(kW)": float(G.max() / DELTA),
201         "储能最大放电功率(kW)": float(D.max() / DELTA),
202         "储电量达到下限的时刻": [time_label(int(t)) for t in np.flatnonzero(np.abs(S - S_MIN
203 ) <= TOL)],
204         "储电量达到上限的时刻": [time_label(int(t)) for t in np.flatnonzero(np.abs(S - S_MAX
205 ) <= TOL)],
206     },
207     "充电来源分摊": {
208         "口径": "按光伏优先直供负载、富余光伏用于充电分摊；不是新增的电能路由约束。",
209         "光伏充电量(kWh)": float(pv_charge.sum()),
210         "外网充电量(kWh)": float(grid_charge.sum()),
211     },
212     "每四小时汇总": blocks,
213 }
214
215 def analyze_initial_energy(result, model):
216     """在最优面上求S_1的上下端点；辅助优化不替换主调度。"""
217     f, A_eq, b_eq, bounds = model
218     cost = float(f @ result.x)
219     face_A = vstack([A_eq, csr_matrix(f[None, :])], format="csr")
220     face_b = np.append(b_eq, cost) # 使用未舍入费用，不能用展示值35118.60。
221     endpoints, solutions = [], []
222     for direction, name in ((1, "最小初始电量"), (-1, "最大初始电量")):
223         objective = np.zeros_like(f)
224         objective[4*N] = direction
225         endpoint = solve_model((objective, face_A, face_b, bounds))
226         G, C, D, W, S = unpack(endpoint.x)
227         residual = float(np.max(np.abs(A_eq @ endpoint.x - b_eq)))
228         bound_error = max(0.0, max(lower - endpoint.x[i] for i, (lower, _) in enumerate(bounds)
229 ),
230                          max(endpoint.x[i] - upper for i, (_, upper) in enumerate(bounds) if
231 upper is not None))
232         cost_error = float(abs(f @ endpoint.x - cost))
233         dual = float(face_b @ endpoint.eqlin.marginals)
234         for i, (lower, upper) in enumerate(bounds):
235             dual += lower * endpoint.lower.marginals[i]
236             if upper is not None:
237                 dual += upper * endpoint.upper.marginals[i]
238         dual_gap = float(abs(objective @ endpoint.x - dual))
239         stationarity = float(np.max(np.abs(objective - face_A.T @ endpoint.eqlin.marginals
240 - endpoint.lower.marginals - endpoint.upper.marginals
241 )))
242         sign_error = max(0.0, -float(endpoint.lower.marginals.min()), float(endpoint.upper.
243 marginals.max()))
244         if max(residual, bound_error, cost_error, dual_gap, stationarity, sign_error) > TOL:
245             raise RuntimeError(f"{name}的最优面或对偶校验未通过")
246         overlap = int(np.count_nonzero((C > TOL) & (D > TOL)))
247         endpoints.append({"端点": name, "初始储电量(kWh)": float(S[0]),
248                          "购电费用(元)": float(f @ endpoint.x), "费用固定误差(元)": cost_error,
249                          "物理等式最大误差(kWh)": residual, "最大越界量(kWh)": float(bound_error)
250 },
251                          "辅助优化原始与对偶目标差(kWh)": dual_gap,
252                          "辅助优化驻点条件最大误差": stationarity,

```

```

247         "辅助优化对偶符号最大误差": sign_error,
248         "同时充放电时段数": overlap})
249     solutions.append(endpoint.x)
250 low, high = (endpoint["初始储电量(kWh)"] for endpoint in endpoints)
251 if high < low - TOL:
252     raise RuntimeError("初始电量区间上下端点顺序异常")
253 _, c_low, d_low, _, _ = unpack(solutions[0])
254 _, c_high, d_high, _, _ = unpack(solutions[1])
255 conflicting = ((c_low > TOL) & (d_high > TOL)) | ((d_low > TOL) & (c_high > TOL))
256 whole_interval_physical = not conflicting.any() and all(e["同时充放电时段数"] == 0 for e in
    endpoints)
257 return {
258     "分析方法": "保留原约束并固定未舍入最优费用，分别最小化和最大化S_1。",
259     "固定最优费用(元)": cost, "下端点(kWh)": low, "上端点(kWh)": high,
260     "区间宽度(kWh)": max(0.0, high-low), "数值核验容差": TOL,
261     "是否存在非退化最优区间": "是" if high-low > TOL else "否(数值精度范围内)",
262     "端点校验": endpoints,
263     "整个区间的互斥可行性": ("通过：两端点无相反运行模式，凸组合仍满足互斥。"
264                               if whole_interval_physical else "尚未证明，不能仅凭LP凸性认定互
    斥。"),
265     "填表所用初始储电量(kWh)": float(result.x[4*N]),
266     "填表策略": "沿用原主解，不用辅助优化的端点策略替换。",
267 }
268
269
270 def export_results(directory, result, price, L, R, audit, validation, initial_energy):
271     G, C, D, W, S = unpack(result.x)
272     directory.mkdir(exist_ok=True)
273     # 保存未舍入的求解结果，避免展示精度破坏能量账目。
274     with (directory / "dispatch.csv").open("w", encoding="utf-8-sig", newline="") as stream:
275         writer = csv.writer(stream)
276         writer.writerow(["时段序号", "开始时间", "结束时间", "电价P(元/kWh)", "负载电量L(kWh)", "
    光伏电量R(kWh)",
277                         "购电量G(kWh)", "充电量C(kWh)", "放电量D(kWh)", "弃光量W(kWh)",
278                         "段首储电量S_t(kWh)", "段末储电量S_{t+1}(kWh)", "购电费用(元)"])
279         for t in range(N):
280             writer.writerow([t+1, time_label(t), time_label(t+1), price[t], L[t], R[t],
281                             G[t], C[t], D[t], W[t], S[t], S[t+1], price[t]*G[t]])
282     audit_labels = {
283         "filename": "数据文件", "sha256": "文件SHA256校验值", "period_count": "时段数量",
284         "price_min": "最低电价(元/kWh)", "price_max": "最高电价(元/kWh)",
285         "load_total_kwh": "全天负载电量(kWh)", "pv_total_kwh": "全天光伏电量(kWh)",
286         "load_energy_max_difference_kwh": "负载电量列最大核验误差(kWh)",
287         "pv_energy_max_difference_kwh": "光伏电量列最大核验误差(kWh)",
288     }
289     validation_labels = {
290         "energy_tolerance_kwh": "电量校验容差(kWh)", "cost_tolerance_yuan": "费用校验容差(元)",
291         "energy_balance_max_abs_kwh": "逐时段能量平衡最大误差(kWh)",
292         "storage_dynamics_max_abs_kwh": "储电递推最大误差(kWh)",
293         "daily_cycle_abs_kwh": "日循环误差(kWh)",
294         "bound_max_violation_kwh": "变量边界最大越界量(kWh)",
295         "daily_charge_discharge_abs_kwh": "全天充放电效率关系误差(kWh)",
296         "daily_energy_ledger_abs_kwh": "全天能量账目误差(kWh)",
297         "objective_recalculation_abs_yuan": "购电费用重算误差(元)",

```

```

298     "dual_objective_yuan": "对偶目标值(元)",
299     "primal_dual_gap_abs_yuan": "原始与对偶目标差(元)",
300     "dual_stationarity_max_abs": "对偶驻点条件最大误差",
301     "simultaneous_charge_discharge_count": "同时充放电时段数",
302 }
303 chinese_validation = {
304     "LP数值校验": "通过" if validation["lp_checks_passed"] else "未通过",
305     "充放电互斥检查": "通过" if validation["mutual_exclusivity_passed"] else "未通过",
306     **{label: validation[key] for key, label in validation_labels.items()},
307     "同时充放电明细": [
308         {"时段序号": row["t"], "开始时间": row["start"], "结束时间": row["end"],
309          "充电量(kWh)": row["C_kwh"], "放电量(kWh)": row["D_kwh"]}
310         for row in validation["simultaneous_charge_discharge_periods"]],
311 }
312 summary = {
313     "当前阶段": "主线性规划及初始储电量最优区间分析",
314     "求解结果": {
315         "最优购电费用(元)": float(price @ G), "全天购电量(kWh)": float(G.sum()),
316         "全天充电量_母线侧(kWh)": float(C.sum()), "全天放电量_母线侧(kWh)": float(D.sum()),
317         "全天弃光量(kWh)": float(W.sum()), "起始储电量S_1(kWh)": float(S[0]),
318         "日末储电量S_145(kWh)": float(S[-1]), "实际最低储电量(kWh)": float(S.min()),
319         "实际最高储电量(kWh)": float(S.max()),
320         "全天储能损耗(kWh)": float((1-ETA_C)*C.sum()+(1/ETA_D-1)*D.sum()),
321     },
322     "结果校验": chinese_validation,
323     "结果分析": analyze_solution(result, price, L, R),
324     "初始储电量最优区间": initial_energy,
325     "结果表口径": {"电量单位": "kWh", "费用单位": "元",
326                    "数值精度": "调度CSV和汇总保留计算精度；正式Excel与论文表建议显示两位小数，
327                    汇总按未舍入值计算",
328                    "时间标签修正": "官方模板首段为0:10-0:20、末段为次日0:00-0:10；输出副本修正
329                    为00:00-00:10至23:50-24:00。"},
330     "输入数据核验": {**{label: audit[key] for key, label in audit_labels.items()},
331                      "电量计算口径": "功率列乘以1/6小时，已有电量列仅作交叉核验"},
332     "模型参数": {
333         "时段长度(小时)": DELTA, "充电效率": ETA_C, "放电效率": ETA_D,
334         "储电量下限(kWh)": S_MIN, "储电量上限(kWh)": S_MAX,
335         "单时段充放电上限_母线侧(kWh)": MAX_ENERGY, "初始储电量": "由优化决定",
336         "日循环约束": "启用", "二进制变量": "未使用", "次级目标": "未使用",
337     },
338     "求解器信息": {
339         "求解方法": "SciPy线性规划接口 / HiGHS", "状态码": int(result.status),
340         "求解状态": "已获得最优解", "迭代次数": int(result.nit),
341         "原始可行性容差": SOLVER_OPTIONS["primal_feasibility_tolerance"],
342         "对偶可行性容差": SOLVER_OPTIONS["dual_feasibility_tolerance"],
343         "变量数量": len(result.x), "等式约束数量": 2*N+1,
344         "Python版本": platform.python_version(), "NumPy版本": np.__version__, "SciPy版本":
345         scipy.__version__,
346     },
347     "结果说明": ("主解通过互斥检查，无需互斥修复；初始电量的唯一性见最优区间分析，结果表仍沿用原
主解。"
                 if validation["mutual_exclusivity_passed"] else
                 "LP通过数值校验，但存在同时充放电；需进入处理阶段，尚不能作为最终物理调度。"),
}

```

```

348     with (directory / "summary.json").open("w", encoding="utf-8") as stream:
349         json.dump(summary, stream, ensure_ascii=False, indent=2, allow_nan=False)
350         stream.write("\n")
351     return summary
352
353
354 def main():
355     base = Path(__file__).resolve().parent
356     price, L, R, audit = load_data(base / "附件1.csv")
357     model = build_lp(price, L, R)
358     result = solve_model(model)
359     validation = validate_solution(result, price, L, R, model)
360     initial_energy = analyze_initial_energy(result, model)
361     summary = export_results(base / "outputs", result, price, L, R, audit, validation,
362                             initial_energy)
363     if not validation["mutual_exclusivity_passed"]:
364         raise RuntimeError("存在同时充放电，已保存调度审查结果；尚不能作为最终物理调度")
365     print(json.dumps({key: summary[key] for key in ("求解结果", "初始储电量最优区间", "结果说明")
366                    },
367                    ensure_ascii=False, indent=2))
367
368 if __name__ == "__main__":
369     main()

```

问题二两阶段随机线性规划的稀疏装配 model.py

该模块给出问题二的物理与价格参数、决策层与执行层的变量布局、稀疏约束装配与求解入口，含充电来源约束与去退化项。

代码 2 问题二两阶段随机线性规划的稀疏装配 model.py

```

1  # -*- coding: utf-8 -*-
2  """问题二两阶段随机线性规划的稀疏装配与求解。
3
4  决策层：第 d 天 0:00 求解，第一阶段变量为对所有情景共享的计划购电量 G，
5  第二阶段变量为逐情景的充放电、储电量、紧急购电量与弃置电量：
6
7      min  sum_t P_t G_t + sum_w pi_w sum_t 5 P_t E_wt
8      s.t. G_t + E_wt + R_wt + D_wt = L_wt + C_wt + W_wt          (能量平衡)
9           S_w,t+1 = S_wt + eta_c C_wt - D_wt / eta_d          (SOC 递推)
10          S_w,1 = S_start
11          C_wt <= max(0, R_wt - L_wt) + G_t                    (充电来源约束)
12          Smin <= S_wt <= Smax, 0 <= C_wt, D_wt <= e_bar
13          G, E, W >= 0
14
15  执行层：固定决策层的 G，把当日实测负载与光伏代入同一组物理约束，
16  以 min sum_t 5 P_t E_t 求解实际执行路径 C, D, S, E, W，即填报 result2 的量。
17  执行层的 G 是常数，因此"负载优先"可以写成精确的上下界：
18
19      C_t <= max(0, R_t + G_t - L_t)          (缺口时段不得充电)
20      E_t <= max(0, L_t - R_t - G_t)          (富余时段不得紧急购电)
21

```

```

22 两条合起来推出  $C_t * E_t = 0$ ，即不存在"一边紧急购电一边充电"的时段。
23
24 **关于充电来源约束的动机。** 仅靠  $G + E + R + D = L + C + W$  一条平衡式，
25 线性规划会允许"用紧急购电支持储能充电"：充电量  $C$  是情景特定变量，计划购电量
26  $G$  是全情景共享变量，于是"在低价时段用该情景的紧急电充电、再到高价时段放电替代
27 紧急购电"成为套利路径。Q2Project.md 的假设 3 曾认为  $5P_t > P_t$  会自动排除它，
28 该判断不成立：跨时段比较的是"低价时段  $5P$ "与"高价时段  $5P$ "，而不是与同时刻的  $P$  比。
29 上述约束把充电能力与共享的计划购电绑定，执行层再以精确上下界落实物理互斥。
30
31 变量索引一律通过 `VariableLayout` 计算，避免分块顺序与索引映射错位。
32 """
33 from __future__ import annotations
34
35 from dataclasses import dataclass
36
37 import numpy as np
38 from scipy.optimize import linprog
39 from scipy.sparse import coo_matrix, csr_matrix
40
41 SLOTS = 144
42 SOLVER_OPTIONS = {"primal_feasibility_tolerance": 1e-10,
43                  "dual_feasibility_tolerance": 1e-10}
44 RESIDUAL_TOL = 1e-6
45
46
47 @dataclass(frozen=True)
48 class ModelParams:
49     """问题二的物理与价格参数，默认值取自 notation.md 与附录 1。
50
51     `degeneracy_penalty` 是去退化项的单价（元/kWh），量级取  $1e-6$ ，比最低电价低六个
52     数量级。线性规划在"紧急购电总量相同"的前提下对充放电路径存在大量等价最优解，
53     不加该项时求解器可能返回"同时充放电"的退化解。该系数只用于在等价最优解中挑出
54     充放电动作最少的路径，不改变主成本与计划购电量。
55     """
56
57     price: np.ndarray
58     delta: float = 1.0 / 6.0
59     s_min: float = 1200.0
60     s_max: float = 10800.0
61     max_power_kw: float = 5000.0
62     eta_c: float = 0.9
63     eta_d: float = 0.9
64     emergency_multiplier: float = 5.0
65     degeneracy_penalty: float = 1e-6
66
67     @property
68     def max_energy(self) -> float:
69         return self.max_power_kw * self.delta
70
71     def __post_init__(self):
72         price = np.asarray(self.price, dtype=float)
73         if price.shape != (SLOTS,) or not np.isfinite(price).all() or (price <= 0).any():
74             raise ValueError("电价必须是 144 维严格正有限向量")
75         object.__setattr__(self, "price", price)

```

```

76
77
78 @dataclass(frozen=True)
79 class LinearProgram:
80     """装配好的线性规划，含等式块、不等式块、边界与变量布局。"""
81
82     c: np.ndarray
83     A_eq: csr_matrix
84     b_eq: np.ndarray
85     A_ub: csr_matrix
86     b_ub: np.ndarray
87     bounds: list
88     layout: object
89     kind: str
90
91     @property
92     def size(self) -> int:
93         return len(self.c)
94
95
96 @dataclass(frozen=True)
97 class VariableLayout:
98     """决策层变量分块与索引映射。
99
100     分块顺序为 [G(144), GC(144), C(K*144), D(K*144), S(K*145), E(K*144), W(K*144)]。
101     GC_t 是"计划购电中划给储能充电的份额"，也是第一阶段变量，因此对所有情景共享。
102     """
103
104     scenarios: int
105
106     @property
107     def n_G(self) -> int:
108         return SLOTS
109
110     @property
111     def offset_G(self) -> int:
112         return 0
113
114     @property
115     def offset_GC(self) -> int:
116         return self.n_G
117
118     @property
119     def offset_C(self) -> int:
120         return self.offset_GC + SLOTS
121
122     @property
123     def offset_D(self) -> int:
124         return self.offset_C + self.scenarios * SLOTS
125
126     @property
127     def offset_S(self) -> int:
128         return self.offset_D + self.scenarios * SLOTS
129

```

```

130     @property
131     def offset_E(self) -> int:
132         return self.offset_S + self.scenarios * (SLOTS + 1)
133
134     @property
135     def offset_W(self) -> int:
136         return self.offset_E + self.scenarios * SLOTS
137
138     @property
139     def size(self) -> int:
140         return self.offset_W + self.scenarios * SLOTS
141
142     def G(self, t: int) -> int:
143         return self.offset_G + t
144
145     def GC(self, t: int) -> int:
146         return self.offset_GC + t
147
148     def C(self, k: int, t: int) -> int:
149         return self.offset_C + k * SLOTS + t
150
151     def D(self, k: int, t: int) -> int:
152         return self.offset_D + k * SLOTS + t
153
154     def S(self, k: int, t: int) -> int:
155         return self.offset_S + k * (SLOTS + 1) + t
156
157     def E(self, k: int, t: int) -> int:
158         return self.offset_E + k * SLOTS + t
159
160     def W(self, k: int, t: int) -> int:
161         return self.offset_W + k * SLOTS + t
162
163
164 @dataclass(frozen=True)
165 class RecourseLayout:
166     """执行层变量分块: G 已定, 其余同决策层单一情景。"""
167
168     @property
169     def size(self) -> int:
170         return 4 * SLOTS + SLOTS + 1
171
172     def C(self, t: int) -> int:
173         return t
174
175     def D(self, t: int) -> int:
176         return SLOTS + t
177
178     def S(self, t: int) -> int:
179         return 2 * SLOTS + t
180
181     def E(self, t: int) -> int:
182         return 3 * SLOTS + 1 + t
183

```

```

184     def W(self, t: int) -> int:
185         return 4 * SLOTS + 1 + t
186
187
188     def validate_scenarios(load: np.ndarray, pv: np.ndarray, weights: np.ndarray) -> tuple:
189         """校验情景矩阵与权重形状、有限性与归一化。"""
190         load, pv, weights = (np.asarray(a, dtype=float) for a in (load, pv, weights))
191         if load.ndim != 2 or load.shape != pv.shape or load.shape[1] != SLOTS:
192             raise ValueError("情景负载与光伏必须是同形状的 (K,144) 矩阵")
193         if weights.shape != (len(load),):
194             raise ValueError("情景权重长度必须等于情景数")
195         if not (np.isfinite(load).all() and np.isfinite(pv).all() and np.isfinite(weights).all()):
196             raise ValueError("情景与权重必须为有限值")
197         if (load < 0).any() or (pv < 0).any() or (weights < 0).any():
198             raise ValueError("情景功率与权重必须非负")
199         if not np.isclose(weights.sum(), 1.0, rtol=0, atol=1e-12):
200             raise ValueError(f"情景权重必须归一化，当前和为 {weights.sum():r}")
201         return load, pv, weights
202
203
204     def build_decision_lp(params: ModelParams, load: np.ndarray, pv: np.ndarray,
205                          weights: np.ndarray, s_start: float) -> LinearProgram:
206         """装配第 d 天的两阶段随机线性规划。
207
208         充电来源用显式的一阶段份额 GC_t 表达：
209
210             C_wt <= max(0, R_wt - L_wt) + GC_t           (充电只能来自光伏富余与计划电份额)
211             GC_t <= G_t - Delta_t                        (计划电先覆盖期望净负荷缺口)
212             0 <= GC_t <= G_t
213
214         其中 Delta_t = sum_w pi_w max(0, L_wt - R_wt) 是该时段的期望净负荷缺口。
215         因为 GC_t 对所有情景共享，规划不能再"用某个情景自己的紧急购电去充电"，
216         紧急购电与充电的时段级互斥由此在决策层也得到保证。
217         """
218         load, pv, weights = validate_scenarios(load, pv, weights)
219         if not (params.s_min <= s_start <= params.s_max):
220             raise ValueError("日初储电量必须落在 [Smin, Smax] 内")
221         scenarios = len(load)
222         layout = VariableLayout(scenarios)
223         net = load - pv
224         deficit = np.maximum(0.0, load - pv)
225         expected_deficit = weights @ deficit
226         rows, cols, data = [], [], []
227         ub_rows, ub_cols, ub_data, ub_bound = [], [], [], []
228
229         def add(row, col, value):
230             rows.append(row)
231             cols.append(col)
232             data.append(value)
233
234         def add_ub(row, col, value):
235             ub_rows.append(row)
236             ub_cols.append(col)
237             ub_data.append(value)

```

```

238
239 # 目标: 计划购电费 + 紧急购电费的经验期望 + 去退化项
240 c = np.zeros(layout.size)
241 c[:layout.n_G] = params.price
242 emergency_price = params.emergency_multiplier * params.price
243 for k in range(scenarios):
244     for t in range(SLOTS):
245         c[layout.E(k, t)] = weights[k] * emergency_price[t]
246         c[layout.C(k, t)] = weights[k] * params.degeneracy_penalty
247         c[layout.D(k, t)] = weights[k] * params.degeneracy_penalty
248
249 # 能量平衡:  $G + E + D - C - W = L - R$ 
250 for k in range(scenarios):
251     for t in range(SLOTS):
252         row = k * SLOTS + t
253         add(row, layout.G(t), 1.0)
254         add(row, layout.E(k, t), 1.0)
255         add(row, layout.D(k, t), 1.0)
256         add(row, layout.C(k, t), -1.0)
257         add(row, layout.W(k, t), -1.0)
258 # SOC 递推:  $S_{t+1} - S_t - \eta C + D / \eta = 0$ 
259 soc_offset = scenarios * SLOTS
260 for k in range(scenarios):
261     for t in range(SLOTS):
262         row = soc_offset + k * SLOTS + t
263         add(row, layout.S(k, t + 1), 1.0)
264         add(row, layout.S(k, t), -1.0)
265         add(row, layout.C(k, t), -params.eta_c)
266         add(row, layout.D(k, t), 1.0 / params.eta_d)
267 # 共同出发点
268 start_offset = 2 * scenarios * SLOTS
269 for k in range(scenarios):
270     add(start_offset + k, layout.S(k, 0), 1.0)
271 # 充电来源约束:  $C_{wt} - GC_t \leq \max(0, R_{wt} - L_{wt})$ 
272 for k in range(scenarios):
273     for t in range(SLOTS):
274         row = k * SLOTS + t
275         add_ub(row, layout.C(k, t), 1.0)
276         add_ub(row, layout.GC(t), -1.0)
277         ub_bound.append(max(0.0, pv[k, t] - load[k, t]))
278 # 计划电份额上限:  $GC_t - G_t \leq -\Delta_t$ , 同时给出  $G_t \geq \Delta_t$ 
279 for t in range(SLOTS):
280     row = scenarios * SLOTS + t
281     add_ub(row, layout.GC(t), 1.0)
282     add_ub(row, layout.G(t), -1.0)
283     ub_bound.append(-float(expected_deficit[t]))
284
285 A_eq = coo_matrix((data, (rows, cols)),
286                  shape=(2 * scenarios * SLOTS + scenarios, layout.size)).tocsr()
287 b_eq = np.concatenate([net.ravel(),
288                        np.zeros(scenarios * SLOTS),
289                        np.full(scenarios, float(s_start))])
290 A_ub = coo_matrix((ub_data, (ub_rows, ub_cols)),
291                  shape=(scenarios * SLOTS + SLOTS, layout.size)).tocsr()

```

```

292     bounds = ([[0.0, None]] * (layout.n_G + SLOTS)
293               + [[0.0, params.max_energy]] * (2 * scenarios * SLOTS)
294               + [[params.s_min, params.s_max]] * (scenarios * (SLOTS + 1))
295               + [[0.0, None]] * (2 * scenarios * SLOTS))
296     return LinearProgram(c, A_eq, b_eq, A_ub, np.asarray(ub_bound), bounds, layout,
297                        "decision")
298
299
300 def build_recourse_lp(params: ModelParams, plan: np.ndarray, load: np.ndarray,
301                      pv: np.ndarray, s_start: float) -> LinearProgram:
302     """固定计划购电量后的实际执行补救问题。"""
303     plan = np.asarray(plan, dtype=float)
304     load, pv = np.asarray(load, dtype=float), np.asarray(pv, dtype=float)
305     if plan.shape != (SLOTS,) or load.shape != (SLOTS,) or pv.shape != (SLOTS,):
306         raise ValueError("计划购电、负载与光伏都必须为 144 维")
307     if (plan < 0).any() or (load < 0).any() or (pv < 0).any():
308         raise ValueError("计划购电量、负载与光伏必须非负")
309     if not (params.s_min <= s_start <= params.s_max):
310         raise ValueError("日初储电量必须落在 [Smin, Smax] 内")
311     layout = RecourseLayout()
312     net = load - pv
313     rows, cols, data = [], [], []
314     ub_rows, ub_cols, ub_data, ub_bound = [], [], [], []
315
316     def add(row, col, value):
317         rows.append(row)
318         cols.append(col)
319         data.append(value)
320
321     def add_ub(row, col, value):
322         ub_rows.append(row)
323         ub_cols.append(col)
324         ub_data.append(value)
325
326     c = np.zeros(layout.size)
327     for t in range(SLOTS):
328         c[layout.E(t)] = params.emergency_multiplier * params.price[t]
329         c[layout.C(t)] = params.degeneracy_penalty
330         c[layout.D(t)] = params.degeneracy_penalty
331     # 能量平衡:  $E + D - C - W = L - R - G$ 
332     for t in range(SLOTS):
333         add(t, layout.E(t), 1.0)
334         add(t, layout.D(t), 1.0)
335         add(t, layout.C(t), -1.0)
336         add(t, layout.W(t), -1.0)
337     for t in range(SLOTS):
338         row = SLOTS + t
339         add(row, layout.S(t + 1), 1.0)
340         add(row, layout.S(t), -1.0)
341         add(row, layout.C(t), -params.eta_c)
342         add(row, layout.D(t), 1.0 / params.eta_d)
343     add(2 * SLOTS, layout.S(0), 1.0)
344     # 负载优先:  $A_t = R_t + G_t - L_t$  为常数
345     surplus = np.maximum(0.0, pv + plan - load)

```

```

346 deficit = np.maximum(0.0, load - pv - plan)
347 for t in range(SLOTS):
348     add_ub(t, layout.C(t), 1.0)
349     ub_bound.append(float(surplus[t]))
350 for t in range(SLOTS):
351     add_ub(SLOTS + t, layout.E(t), 1.0)
352     ub_bound.append(float(deficit[t]))
353
354 A_eq = coo_matrix((data, (rows, cols)), shape=(2 * SLOTS + 1, layout.size)).tocsr()
355 b_eq = np.concatenate([net - plan, np.zeros(SLOTS), [float(s_start)]])
356 A_ub = coo_matrix((ub_data, (ub_rows, ub_cols)),
357                   shape=(2 * SLOTS, layout.size)).tocsr()
358 bounds = [(0.0, params.max_energy)] * (2 * SLOTS)
359           + [(params.s_min, params.s_max)] * (SLOTS + 1)
360           + [(0.0, None)] * (2 * SLOTS)
361 return LinearProgram(c, A_eq, b_eq, A_ub, np.asarray(ub_bound), bounds, layout,
362                    "recourse")
363
364
365 def solve(program: LinearProgram, tag: str):
366     """调用 SciPy 的 HiGHS 接口求解，非最优直接报错而不静默返回。"""
367     result = linprog(program.c, A_ub=program.A_ub, b_ub=program.b_ub,
368                     A_eq=program.A_eq, b_eq=program.b_eq, bounds=program.bounds,
369                     method="highs", options=SOLVER_OPTIONS)
370     if not result.success or result.status != 0:
371         raise RuntimeError(f"{tag} 未达到最优状态: {result.message}")
372     if not np.isfinite(result.x).all():
373         raise RuntimeError(f"{tag} 的解包含非有限值")
374     return result
375
376
377 def unpack_decision(result, layout: VariableLayout, scenarios: int) -> dict:
378     """按布局拆解决策层解向量，返回逐情景数组与第一阶段计划。"""
379     x = result.x
380     G = x[layout.offset_G:layout.offset_G + layout.n_G]
381     shape = (scenarios, SLOTS)
382     return {
383         "G": G,
384         "GC": x[layout.offset_GC:layout.offset_GC + SLOTS],
385         "C": x[layout.offset_C:layout.offset_C + scenarios * SLOTS].reshape(shape),
386         "D": x[layout.offset_D:layout.offset_D + scenarios * SLOTS].reshape(shape),
387         "S": x[layout.offset_S:layout.offset_S + scenarios * (SLOTS + 1)].reshape(
388             (scenarios, SLOTS + 1)),
389         "E": x[layout.offset_E:layout.offset_E + scenarios * SLOTS].reshape(shape),
390         "W": x[layout.offset_W:layout.offset_W + scenarios * SLOTS].reshape(shape),
391         "objective": float(result.fun),
392     }
393
394
395 def unpack_recourse(result, layout: RecourseLayout) -> dict:
396     """按布局拆解执行层解向量。"""
397     x = result.x
398     return {"C": x[:SLOTS], "D": x[SLOTS:2 * SLOTS], "S": x[2 * SLOTS:3 * SLOTS + 1],
399           "E": x[3 * SLOTS + 1:4 * SLOTS + 1], "W": x[4 * SLOTS + 1:5 * SLOTS + 1],

```

调度优化层 `dispatch.py`

该模块给出规划层的两阶段随机线性规划与执行层的确定性调度，对应问题三中对模型预测控制的两种求解口径。

代码 3 调度优化层 `dispatch.py`

```

1  """调度优化层（对齐 problem3_solving_final.md 式 16-29）。
2
3  - 规划层 solve_planning: 两阶段 wait-and-see 随机 LP, 单 W 能量平衡。
4  - 执行层 solve_dispatch: 确定性单情景 LP/MILP, 式 23 充放电/紧急购电互斥。
5  """
6  import numpy as np
7
8  import myh.src.config as config
9  from myh.src.lp import Model
10
11  INF = np.inf
12
13
14  def solve_dispatch(load, pv, price, s0, G_fixed=None, s_end=None,
15                  term_a=None, term_b=None, mutex=False):
16      """执行层确定性调度（式 22-23）。单 W 平衡，可选当前区间互斥二元 z,y。"""
17      H = len(load)
18      m = Model()
19      free_G = G_fixed is None
20
21      G = [m.var(f"G_{i}", lb=0.0, obj=0.0 if not free_G else price[i])
22           for i in range(H)]
23      C = [m.var(f"C_{i}", lb=0.0, ub=config.E_BAR) for i in range(H)]
24      D = [m.var(f"D_{i}", lb=0.0, ub=config.E_BAR) for i in range(H)]
25      E = [m.var(f"E_{i}", lb=0.0, obj=config.EMERGENCY_MULT * price[i]) for i in range(H)]
26      W = [m.var(f"W_{i}", lb=0.0) for i in range(H)]
27      S = [m.var(f"S_{i}", lb=config.S_MIN, ub=config.S_MAX) for i in range(H + 1)]
28      theta = m.var("theta", lb=0.0, obj=1.0) if term_a is not None else None
29
30      if mutex and H > 0:
31          z = m.var("z", lb=0.0, ub=1.0, integer=True)
32          y = m.var("y", lb=0.0, ub=1.0, integer=True)
33          m.add({C[0]: 1.0, z: -config.E_BAR}, 'le', 0.0)
34          m.add({D[0]: 1.0, z: config.E_BAR}, 'le', config.E_BAR)
35          m.add({E[0]: 1.0, y: -load[0]}, 'le', 0.0)
36          m.add({C[0]: 1.0, y: config.E_BAR}, 'le', config.E_BAR)
37
38      m.add({S[0]: 1.0}, 'eq', s0)
39      if s_end is not None:
40          m.add({S[H]: 1.0}, 'eq', s_end)
41      if term_a is not None:
42          for q in range(len(term_a)):
43              m.add({theta: 1.0, S[H]: -term_a[q]}, 'ge', term_b[q])
44

```

```

45     for i in range(H):
46         terms = {E[i]: 1.0, D[i]: 1.0, C[i]: -1.0, W[i]: -1.0}
47         rhs = load[i] - pv[i]
48         if free_G:
49             terms[G[i]] = 1.0
50         else:
51             rhs -= G_fixed[i]
52         m.add(terms, 'eq', rhs)
53         m.add({S[i + 1]: 1.0, S[i]: -1.0, C[i]: -config.ETA_C,
54               D[i]: 1.0 / config.ETA_D}, 'eq', 0.0)
55
56     status, msg, x = m.solve()
57     if status != 'ok':
58         return {"status": status, "message": msg}
59     Cv = np.array([x[c] for c in C])
60     Dv = np.array([x[d] for d in D])
61     Ev = np.array([x[e] for e in E])
62     Wv = np.array([x[w] for w in W])
63     Sv = np.array([x[s] for s in S])
64     Gv = np.array([x[g] for g in G]) if free_G else np.asarray(G_fixed, float)
65     obj = float(np.sum(price * Gv + config.EMERGENCY_MULT * price * Ev))
66     if theta is not None:
67         obj += float(x[theta])
68     return {"status": "ok", "C": Cv, "D": Dv, "E": Ev, "W": Wv, "S": Sv,
69           "G": Gv, "obj": obj}
70
71
72 def solve_planning(L, R, p, probs, s0, GO=None, term_a=None, term_b=None,
73                   use_cvar=False, alpha=0.95, lam=0.0, g_floor_q=None):
74     """两阶段随机 LP (式 16/21/27/29) 。
75
76     L/R/p: (n_scen, H) 情景负载/光伏/电价; probs: (n_scen,) 等权。
77     第一阶段 G (here-and-now)、A (调整量) 全情景共用; 第二阶段 C/D/E/W/S/ 按情景。
78     """
79     L, R, p = [np.asarray(x, float) for x in (L, R, p)]
80     probs = np.asarray(probs, float)
81     N, H = L.shape
82
83     # 合并完全重复情景 (同一历史块副本): 概率相加, 分布不变。
84     if N > 1:
85         stacked = np.concatenate([L, R, p], axis=1)
86         uniq, first_idx, inverse = np.unique(stacked, axis=0,
87                                             return_index=True, return_inverse=True)
88         if uniq.shape[0] < N:
89             L, R, p = L[first_idx], R[first_idx], p[first_idx]
90             probs = np.bincount(inverse, weights=probs)
91             N, H = L.shape
92
93     pbar = probs @ p
94     has_adj = GO is not None
95     scale = (1.0 - lam) if use_cvar else 1.0
96
97     # 报童下界: G 不得低于情景净负荷的 g_floor_q 分位 (对冲 wait-and-see 乐观偏差)
98     net_floor = None

```

```

99     if g_floor_q is not None:
100         net_floor = np.maximum(0.0, np.quantile(L - R, g_floor_q, axis=0))
101
102     m = Model()
103     G = [m.var(f"G_{i}", lb=0.0 if net_floor is None else net_floor[i],
104             obj=scale * pbar[i]) for i in range(H)]
105     A = [m.var(f"A_{i}", lb=0.0,
106             obj=scale * 0.5 * pbar[i] if has_adj else 0.0) for i in range(H)]
107
108     C = [[None] * H for _ in range(N)]
109     D = [[None] * H for _ in range(N)]
110     E = [[None] * H for _ in range(N)]
111     W = [[None] * H for _ in range(N)]
112     S = [[None] * (H + 1) for _ in range(N)]
113     theta = [None] * N
114
115     for w in range(N):
116         pw = probs[w]
117         for i in range(H):
118             C[w][i] = m.var(f"C_{w}_{i}", lb=0.0, ub=config.E_BAR)
119             D[w][i] = m.var(f"D_{w}_{i}", lb=0.0, ub=config.E_BAR)
120             E[w][i] = m.var(f"E_{w}_{i}", lb=0.0,
121                             obj=scale * pw * config.EMERGENCY_MULT * p[w, i])
122             W[w][i] = m.var(f"W_{w}_{i}", lb=0.0)
123         for i in range(H + 1):
124             S[w][i] = m.var(f"S_{w}_{i}", lb=config.S_MIN, ub=config.S_MAX)
125         theta[w] = m.var(f"theta_{w}", lb=0.0, obj=scale * pw)
126
127     m.add({S[w][0]: 1.0}, 'eq', s0)
128     if term_a is not None:
129         for q in range(len(term_a)):
130             m.add({theta[w]: 1.0, S[w][H]: -term_a[q]}, 'ge', term_b[q])
131     for i in range(H):
132         # 式 16 单 W 能量平衡
133         m.add({G[i]: 1.0, E[w][i]: 1.0, D[w][i]: 1.0,
134             C[w][i]: -1.0, W[w][i]: -1.0}, 'eq', L[w, i] - R[w, i])
135         m.add({S[w][i + 1]: 1.0, S[w][i]: -1.0,
136             C[w][i]: -config.ETA_C, D[w][i]: 1.0 / config.ETA_D}, 'eq', 0.0)
137
138     if has_adj:
139         for i in range(H):
140             m.add({A[i]: 1.0, G[i]: -1.0}, 'ge', -GO[i])
141             m.add({A[i]: 1.0, G[i]: 1.0}, 'ge', GO[i])
142
143     if use_cvar:
144         zeta = m.var("zeta", lb=-INF, obj=lam)
145         u = [m.var(f"u_{w}", lb=0.0, obj=lam * probs[w] / (1.0 - alpha))
146             for w in range(N)]
147         for w in range(N):
148             terms = {u[w]: 1.0, zeta: 1.0, theta[w]: -1.0}
149             for i in range(H):
150                 terms[f"G_{i}"] = -p[w, i]
151             if has_adj:
152                 terms[f"A_{i}"] = -0.5 * p[w, i]

```

```

153         terms[f"E_{w}_{i}"] = -config.EMERGENCY_MULT * p[w, i]
154     m.add(terms, 'ge', 0.0)
155
156     status, msg, x = m.solve()
157     if status != 'ok':
158         return {"status": status, "message": msg}
159
160     Gv = np.array([x[g] for g in G])
161     Av = np.array([x[a] for a in A])
162     Eemg = sum(probs[w] * config.EMERGENCY_MULT *
163               float(np.sum(p[w] * np.array([x[e] for e in E[w]]))))
164               for w in range(N))
165     Eadj = 0.5 * float(np.sum(pbar * Av)) if has_adj else 0.0
166     obj = float(np.sum(pbar * Gv)) + Eemg + Eadj
167     return {"status": "ok", "G": Gv, "A": Av, "obj": obj}
168
169
170 def solve_mpc(L, R, p, probs, s0, G_fixed, term_a=None, term_b=None, mutex=True):
171     """执行层情景 MPC: 当前区间 here-and-now (全情景共用), 未来 wait-and-see.
172
173     L/R/p: (n_scen, H)。当前区间 t=0 的 C/D/E/W 全情景共用 (可实施动作);
174     t>0 按情景自由补救。G_fixed 为已提交、不可改的购电计划。mutex 施加式 23 互斥。
175     """
176     L, R, p = [np.asarray(x, float) for x in (L, R, p)]
177     probs = np.asarray(probs, float)
178     N, H = L.shape
179
180     # 合并完全重复情景
181     if N > 1:
182         stacked = np.concatenate([L, R, p], axis=1)
183         uniq, first_idx, inverse = np.unique(stacked, axis=0,
184                                             return_index=True, return_inverse=True)
185         if uniq.shape[0] < N:
186             L, R, p = L[first_idx], R[first_idx], p[first_idx]
187             probs = np.bincount(inverse, weights=probs)
188             N, H = L.shape
189
190     m = Model()
191
192     # 当前区间 (here-and-now, 全情景共用)
193     CO = m.var("CO", lb=0.0, ub=config.E_BAR)
194     DO = m.var("DO", lb=0.0, ub=config.E_BAR)
195     EO = m.var("EO", lb=0.0,
196               obj=config.EMERGENCY_MULT * float(np.sum(probs * p[:, 0])))
197     WO = m.var("WO", lb=0.0)
198
199     # 未来区间 (wait-and-see)
200     C = [[None] * H for _ in range(N)]
201     D = [[None] * H for _ in range(N)]
202     E = [[None] * H for _ in range(N)]
203     W = [[None] * H for _ in range(N)]
204     S = [[None] * (H + 1) for _ in range(N)]
205     theta = [None] * N
206

```

```

207     for w in range(N):
208         pw = probs[w]
209         for t in range(1, H):
210             C[w][t] = m.var(f"C_{w}_{t}", lb=0.0, ub=config.E_BAR)
211             D[w][t] = m.var(f"D_{w}_{t}", lb=0.0, ub=config.E_BAR)
212             E[w][t] = m.var(f"E_{w}_{t}", lb=0.0,
213                             obj=pw * config.EMERGENCY_MULT * p[w, t])
214             W[w][t] = m.var(f"W_{w}_{t}", lb=0.0)
215         for t in range(H + 1):
216             S[w][t] = m.var(f"S_{w}_{t}", lb=config.S_MIN, ub=config.S_MAX)
217             theta[w] = m.var(f"theta_{w}", lb=0.0, obj=pw)
218
219         m.add({S[w][0]: 1.0}, 'eq', s0)
220         if term_a is not None:
221             for q in range(len(term_a)):
222                 m.add({theta[w]: 1.0, S[w][H]: -term_a[q]}, 'ge', term_b[q])
223
224         # t=0 平衡 (当前区间 L/R 为实测, 全情景相同, 用情景 0 代表)
225         m.add({EO: 1.0, DO: 1.0, CO: -1.0, W0: -1.0}, 'eq',
226               L[0, 0] - R[0, 0] - G_fixed[0])
227         m.add({S[w][1]: 1.0, S[w][0]: -1.0, CO: -config.ETA_C,
228               DO: 1.0 / config.ETA_D}, 'eq', 0.0)
229
230         for t in range(1, H):
231             m.add({E[w][t]: 1.0, D[w][t]: 1.0, C[w][t]: -1.0, W[w][t]: -1.0},
232                   'eq', L[w, t] - R[w, t] - G_fixed[t])
233             m.add({S[w][t + 1]: 1.0, S[w][t]: -1.0, C[w][t]: -config.ETA_C,
234                   D[w][t]: 1.0 / config.ETA_D}, 'eq', 0.0)
235
236     # 式 23: 当前区间互斥 (z 充放电、y 紧急购电不充电)
237     if mutex:
238         z = m.var("z", lb=0.0, ub=1.0, integer=True)
239         y = m.var("y", lb=0.0, ub=1.0, integer=True)
240         m.add({CO: 1.0, z: -config.E_BAR}, 'le', 0.0)
241         m.add({DO: 1.0, z: config.E_BAR}, 'le', config.E_BAR)
242         m.add({EO: 1.0, y: -L[0, 0]}, 'le', 0.0)
243         m.add({CO: 1.0, y: config.E_BAR}, 'le', config.E_BAR)
244
245     status, msg, x = m.solve()
246     if status != 'ok':
247         return {"status": status, "message": msg}
248
249     Cv = np.empty(H)
250     Dv = np.empty(H)
251     Ev = np.empty(H)
252     Wv = np.empty(H)
253     Cv[0], Dv[0], Ev[0], Wv[0] = x[CO], x[DO], x[EO], x[W0]
254     for t in range(1, H):
255         Cv[t] = x[C[0][t]]
256         Dv[t] = x[D[0][t]]
257         Ev[t] = x[E[0][t]]
258         Wv[t] = x[W[0][t]]
259     obj = float(np.sum(probs * np.array([
260         config.EMERGENCY_MULT * (p[w, 0] * x[EO] +

```

```

261         np.sum(p[w, 1:] * np.array([x[E[w][t]] for t in range(1, H)]))
262     )
263     + x[theta[w]] for w in range(N]]))
264 return {"status": "ok", "C": Cv, "D": Dv, "E": Ev, "W": Wv,
        "S": np.array([x[s] for s in S[0]]), "obj": obj}

```